UNIVERSIDAD AUTÓNOMA DE GUADALAJARA

CON RECONOCIMIENTO DE VALIDEZ OFICIAL DE ESTUDIOS DE LA SECRETARÍA DE EDUCACIÓN PÚBLICA SEGÚN ACUERDO No.158 DE FECHA 17 DE JULIO DE 1991

POSGRADO E INVESTIGACIÓN



Servidor LoRa para redes inalámbricas de sensores de largo alcance y baja potencia

TESIS

QUE PRESENTA

JOSÉ FRANCISCO ROSALES FONSECA

PARA OBTENER EL GRADO DE

MAESTRÍA EN CIENCIAS COMPUTACIONALES

DIRECTOR DE TESIS:

DRA LINA MARIA AGUILAR LOBO

GUADALAJARA, JALISCO. AGOSTO 2023

Número de registro en el sistema de investigación

POSTGRADO E INVESTIGACIÓN

Carta de aceptación de proyecto de titulación

Nombre del alumno:
Programa:
Fecha:

Por el presente le informo que su propuesta de titulación ***(Nombre de su trabajo),*** ha sido evaluado por el comité de titulación y de acuerdo a los lineamientos y requisitos institucionales y propios del programa, resultó aprobado para su entrega final y titulación de su postgrado, por lo que se le autoriza continuar con los trámites correspondientes para la presentación de su examen.

Atentamente
“Ciencia y Libertad”

_____	_____
Nombre y firma del representante del Comité de titulación	Nombre y firma del director de la escuela o del programa

Dedicatoria

Dedico este trabajo a mi madre Angelica, por ser el pilar más importante en mi vida y por demostrarme siempre su cariño y apoyo incondicional sin importar nuestras diferencias de opiniones. A mis hermanas Dana y Angelica, que siempre han estado a mi lado apoyándome. A mis amigos Jahir, Alex, Yoshua y Shannen que siempre han creído en mi y me han impulsado a volverme una mejor persona y a perseguir mis sueños.

Los sueños comienzan cuando se cree en ellos.

Declaración expresa

La responsabilidad por los hechos, ideas y doctrinas expuestas en este proyecto corresponden exclusivamente a los directamente involucrados en su desarrollo, y el patrimonio intelectual de la misma a la Universidad Autónoma de Guadalajara.

Agradecimientos

Antes que nada, agradezco a mi madre Angelica por todo el apoyo que siempre me ha brindado en todo el transcurso de mi vida militar, mi vida civil, durante mis estudios, mi carrera profesional y en lo largo de mi vida, sin ella este trabajo nunca hubiera sido posible.

También agradezco a mis amigos y hermanos Jahir, Alex, Yoshua, Dana y Shannen por todo el apoyo que de alguna u otra forma siempre me han dado, en mis momentos más bajos asi como en los más altos gracias por estar ahí. Llevo en el corazón sus palabras de aliento y animo hacia mí.

Cuando ya el dolor te agobie y no puedas más seguir, descansar acaso debes, pero nunca desistir.

Índice general

Siglas **10**

1. Introducción **11**

1.1. Descripción del problema 11

1.2. Definición del problema 11

1.3. Objetivos del proyecto 12

1.3.1. Objetivo General 12

1.4. Delimitación 12

1.5. Justificación 13

1.6. Hipótesis 13

2. Estado del Arte y Bases Teóricas **14**

2.1. Antecedentes 14

2.1.1. Puerta de acceso IoT con Computación de frontera para la Monitorización Agrícola 14

2.1.2. Evaluación experimental de sensores en un sistema de seguimiento GPS basado en LoRaWAN 17

2.1.3. Implementación de LoRaWAN desde el dispositivo final hasta la aplicación 18

2.1.4. Adquisición flexible de datos mediante LoRaWAN y MQTT para pequeñas y medianas empresas 19

2.1.5. Diseño de una pasarela LoRaWAN con servidores autónomos para comunidades sin conexión a Internet 23

2.1.6. Particionado de red de acceso radio en redes LoRaWAN . . . 25

2.1.7. Diseño de un Nodo LoRa-GPS para Localización de Bicicletas Implementado en una Red LoRaWAN 27
2.2. Marco teórico 28
2.2.1. LoRa y LoRaWAN 28
2.2.2. Docker 30
2.2.3. The Things Network 31

3. Metodología **33**
3.1. Diseño del servidor LoRa 33
3.2. Diseño de arquitectura implementado 33
3.3. Configuración del servidor LoRa 34
3.3.1. Prerrequisitos 34
3.3.2. Instalación de Docker 34
3.3.3. Instalación de The Things Stack 35
3.3.4. Configuración de certificados 36
3.3.5. Configurar LAIRD Sentrius RG191 39
3.3.6. Configurar imágenes de Docker 43
3.4. Configurar panel web 44
3.4.1. Registrar puerta de enlace 45
3.4.2. Crear una aplicación 51
3.4.3. Registrar un nuevo dispositivo 54
3.4.4. Leer información usando Linux 60
3.4.5. Leer información usando código en Python 61
3.4.6. Problemas comunes encontrados y sus soluciones 65

4. Conclusiones **68**
4.1. Conclusiones y las recomendaciones 68

Bibliografía **69**

Índice de figuras

2.1. Relación entre la nube, computación de frontera y terminales 15
2.2. Arquitectura de la puerta de enlace o computación de frontera 16
2.3. Arquitectura de la aplicación 18
2.4. Jerarquía de componentes LoRa 19
2.5. Arquitectura general propuesta 20
2.6. Arquitectura general propuesta 20
2.7. Arquitectura MQTT propuesta 21
2.8. Arquitectura LoRa propuesta 21
2.9. Plataforma IoT propuesta 22
2.10. Partes de una Estación Base 23
2.11. BS con 8 canales (izquierda), 1 canal (centro) y un nodo final (derecha). 25
2.12. Arquitectura utilizada para el particionado de la red LoRaWAN . . . 26
2.13. Esquema funcional general de localización de bicicletas con LoRaWAN 27
2.14. Arquitectura LoRa 29
2.15. Topología LoRa 30
2.16. Mapa con la distribución de las puertas de enlace en el mundo para The Things Network 32

3.1. Arquitectura implementada para el sistema de nodos 34
3.2. Árbol de archivos 39
3.3. Página principal del RG191 40
3.4. Modificación de IP Estática en el RG191 41
3.5. Modificación del protocolo LoRa en el RG191 42
3.6. Interfaz enp3s0 43
3.7. Página principal de The Things Stack 45
3.8. Icono de la sección *gateways* en el panel web 46

3.9. Sección con el listado de todos los *gateways* en el panel web 46
3.10. Sección superior de configuración para registro de un *gateway* 47
3.11. Sección inferior de configuración para registro de un *gateway* 48
3.12. Vista de un *gateway* configurado 49
3.13. Panel para desactivación de autenticación 50
3.14. Trafico de la red interceptado en el *gateway* 51
3.15. Icono de la sección *applications* en el panel web 52
3.16. Sección con el listado de todos los *applications* en el panel web 52
3.17. Sección de configuración para registro de un *application* 53
3.18. Vista de una *application* configurada 54
3.19. Sección superior de configuración para registro de un *device* 55
3.20. Sección superior de configuración para registro de un *device* 56
3.21. Sección de frecuencia de configuración para registro de un *device* . . . 56
3.22. Sección media de configuración para registro de un *device* 57
3.23. Sección inferior de configuración para registro de un *device* 58
3.24. Vista de un *device* configurado 59
3.25. Vista de un *device* configurado 60
3.26. Vista de configuración para la IP de un *gateway* 66
3.27. Vista para el formato de los paquetes 67

Siglas

C

Computación de frontera Es un tipo de informática que ocurre en la ubicación física del usuario, de la fuente de datos, o cerca de ellas. 11, 14

Contenedor Un contenedor es un paquete estandarizado de software que se puede implementar de forma rápida y confiable en la nube y otros entornos para admitir una innovación rápida y eficaz. Los contenedores permiten a los desarrolladores empaquetar una aplicación con todas las partes que necesita, como bibliotecas y otras dependencias, y enviarla todo como un paquete. 27

D

Docker Es una plataforma de software de código abierto para crear, implementar y administrar contenedores de aplicaciones virtualizados en un sistema operativo (SO) común, con un ecosistema de herramientas aliadas 27

G

guía oficial https://www.debian.org/releases/buster/amd64/ 31

L

LoRa Tecnología de modulación de radio de largo alcance y baja potencia utilizada como capa física en redes LoRaWAN 9, 10

LoRaWAN Protocolo y arquitectura de red de área amplia y bajo consumo que utiliza LoRa como tecnología de comunicación física entre los dispositivos finales y la infraestructura de red 11

Capítulo 1

Introducción

1.1. Descripción del problema

En una red inalámbrica de sensores de largo alcance y baja potencia, específicamente en aplicaciones de Internet de las cosas (IoT) para ciudades inteligentes se tiene la necesidad de comunicar múltiples microcontroladores para la creación de una red de dispositivos que midan variables físicas en tiempo real y envíen esta información recolectada a un centro de control ubicado remotamente. Debido a la necesidad de un punto central que recabe todos estos datos es necesario la implementación de un servidor maestro el cual debe ser escalable, sencillo de configurar y a prueba de fallos. En este contexto, es igual de importante, contar con una documentación detallada en caso de enfrentar errores comunes o la necesidad de cambiar configuraciones, para que el servidor pueda ser restablecido bajo cualquier circunstancia de falla.

Por lo tanto, se requiere una solución a la necesidad de generar y configurar un servidor físico que funcione a manera de acceso entre un enrutador RG191 (de tecnología LoRa) y un microcontrolador STM32 núcleo L073RZ, así como un manual detallado de como replicarlo en caso de requerirlo.

1.2. Definición del problema

Diversos proyectos de Internet de las cosas para aplicaciones de ciudades inteligentes para realizar pruebas de concepto en un ambiente controlado, no cuentas con

un servidor maestro para el enrutamiento de la red de dispositivos IoT.

1.3. Objetivos del proyecto

1.3.1. Objetivo General

Implementar un servidor Linux que funcione a manera de puente entre un enrutador RG191 de tecnología LoRa y una red de sensores basada en un microcontrolador STM32 núcleo L073RZ, y crear un manual detallado de configuración.

Objetivos específicos:

- Configurar un servidor con un sistema operativo Linux así como todas las herramientas necesarias para su uso como puente entre enrutadores y nodos IoT utilizando el protocolo LoRaWAN.
- Configurar un enrutador para el manejo y gestión de múltiples nodos LoRa tales como el alta y baja de dispositivos.
- Generar un manual detallado que abarque todos los problemas encontrados y el procedimiento para resolverlos.

1.4. Delimitación

El alcance de este proyecto pasa por la generación de documento que cuente con las instrucciones necesarias para replicar la generación de servidores puentes, así como más enrutadores en caso de que la red se quiera escalar en un futuro, así como su manejo interno. La validación del documento se hará experimentalmente en el lanzamiento de la aplicación.

El aporte del proyecto se puede resumir en dos elementos centrales. El primero es la generación de un documento que recopile todos los problemas encontrados, dudas más comunes, instrucciones generales y específicas, así como configuraciones de los dispositivos involucrados. El segundo es la configuración de un servidor físico así como de un enrutador para el manejo y gestión de los nodos LoRa.

1.5. Justificación

Los motivos que llevaron a la necesidad de crear un servidor maestro para el control de nodos LoRa, es la falta de una centralización y control de los nodos IoT comunicados mediante la tecnología de largo alcance LoRa, los cuales son usados para sistemas IoT con aplicaciones para ciudades inteligentes. Debido a la escala del proyecto y la búsqueda de una forma de control centralizada de los mismos, es imperativo el desarrollo de esta solución para un control efectivo de datos y nodos, así como visualización de datos.

1.6. Hipótesis

Contar con un servidor Linux configurado con todas las herramientas necesarias para el manejo de nodos LoRa, así como el uso de estas a través de contenedores para así ser tolerante a fallos, así mismo un manual detallado que describe todos los pasos sobre como configurar y restablecer este sistema con los problemas más comunes encontrados. De esta forma ahorrando tiempo para futuros desarrolladores o investigadores que busquen continuar trabajando en este proyecto.

Capítulo 2

Estado del Arte y Bases Teóricas

2.1. Antecedentes

En esta sección se describen diversos proyectos, artículos científicos, así como proyectos de tesis que tengan relación con el desarrollo de la red LoRaWAN o en otras tecnologías similares.

2.1.1. Puerta de acceso IoT con Computación de frontera para la Monitorización Agrícola

Este artículo científico [1] habla sobre el diseño de una puerta de acceso basado en Computación de frontera para el monitoreo del crecimiento de cultivos. Para esto se utiliza tecnología de contenedores virtualizados para empaquetar todas las herramientas necesarias para el uso del protocolo LoRaWAN en un servidor, herramientas de identificación de pestes e información del entorno a través de funciones que conviertan datos en imágenes. El puente de acceso aplica una capa intermedia de software orientada al uso de mensajes para estandarizar y personalizar la transmisión de datos entre funciones, nubes y fronteras. A través de la simulación y datos de campo el diseño de la puerta de acceso puede obtener las funciones y datos de cada módulo al mismo tiempo, así como la utilización y transmisión con una gran calidad y estabilidad. A continuación en la Figura 2.1 una imagen demostrando la relación entre la nube, Computación de frontera y las terminales:

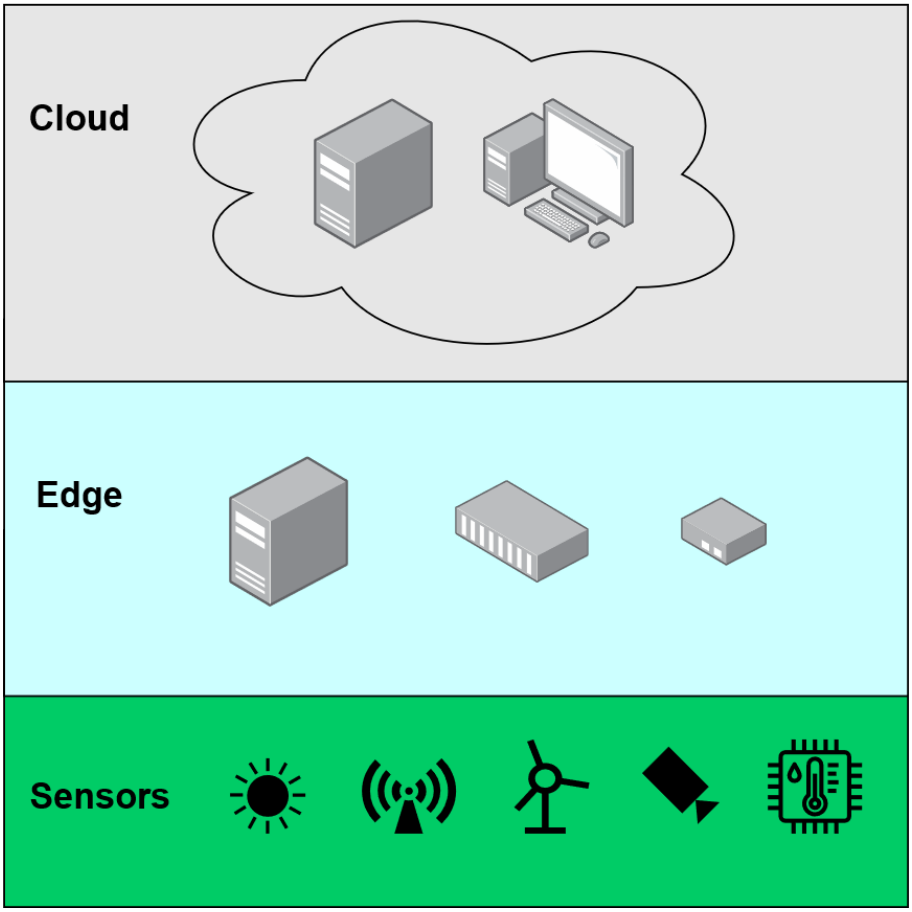


Figura 2.1: Relación entre la nube, computación de frontera y terminales

A si mismo, en la Figura 2.2 se muestra una imagen con la arquitectura de la puerta de enlace:

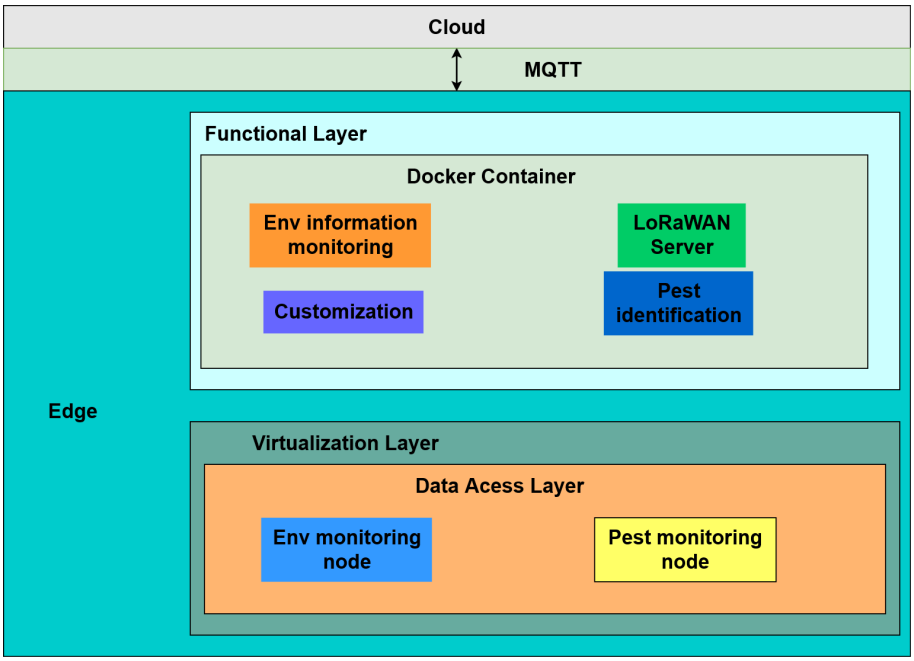


Figura 2.2: Arquitectura de la puerta de enlace o computación de frontera

Los resultados de la investigación demostraron que Computación de frontera tiene la ventaja de ser de bajo costo, baja latencia y bajo consumo de poder lo que la vuelve con gran potencial para uso práctico. Así mismo el apoyo de tecnologías de contenerización y la computación en la nube vuelven al proyecto mas robusto frente a errores y la búsqueda de replicación o escalación de estas funcionalidades y se reportan terminos de reportan una tasa media de éxito en la transmisión de datos del 99,1 por ciento, con el objetivo de reducir la latencia y la carga de los servidores en la nube.

2.1.2. Evaluación experimental de sensores en un sistema de seguimiento GPS basado en LoRaWAN

El siguiente artículo [2] científico se enfoca en proveer una solución basada en LoRaWAN para el desarrollo de ciudades inteligentes con el uso de sensores en un sistema de seguimiento GPS basado en LoRaWAN, una tecnología de comunicación inalámbrica de bajo consumo y largo alcance. Su objetivo es analizar el funcionamiento de los sensores y el rendimiento del sistema para la localización y transmisión de datos, considerando las características de esta tecnología en aplicaciones de seguimiento remoto e IoT. El artículo resalta los beneficios de esta tecnología como es su bajo costo y poder de consumo comparado con distintas alternativas. A través de esta investigación el equipo de desarrollo implementa un sistema de rastreo de una flota de vehículos utilizando LoRaWAN usando como caso de estudio la ciudad de Kozani en Grecia. La arquitectura es utilizada de la siguiente forma en la Figura 2.3. Como se puede ver esta es ligeramente distinta al artículo anterior, pero cabe destacar que es bastante estándar dentro de los modelos LoRa como veremos mas adelante en el marco teórico.

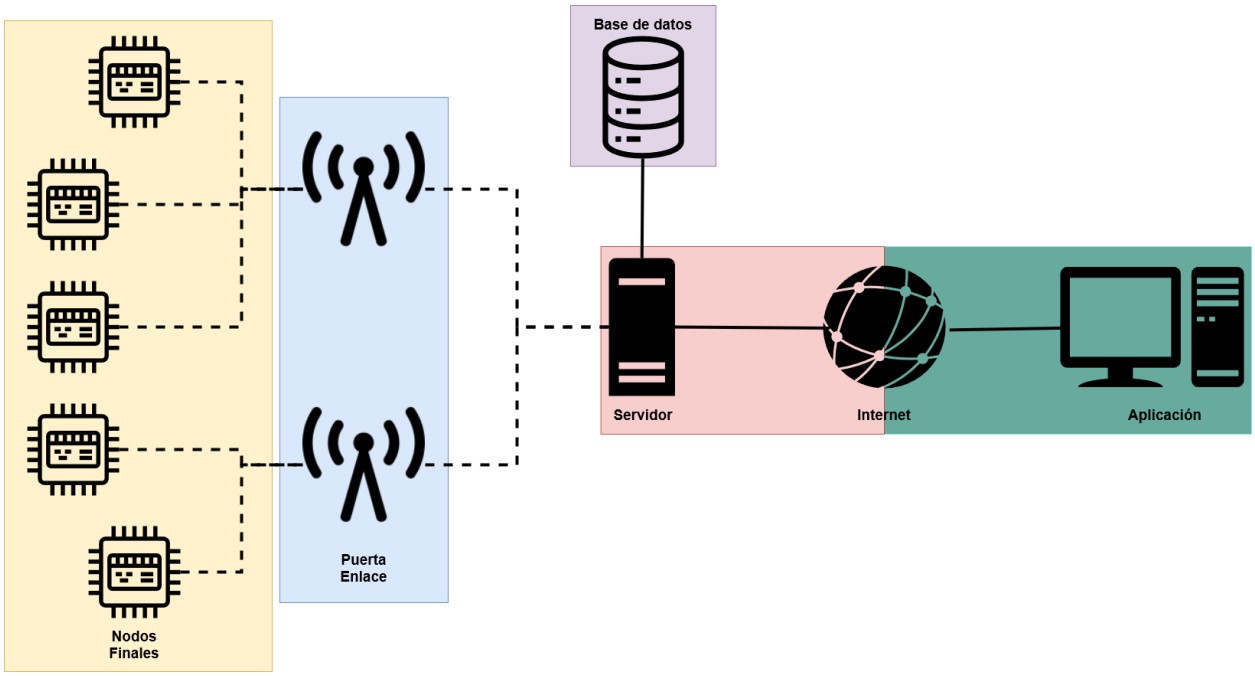


Figura 2.3: Arquitectura de la aplicación

De izquierda a derecha la arquitectura muestra un sistema de nodos finales basados en antenas GPS comunicándose con puertas de enlace a través de LoRaWAN, estos últimos a su vez redirigen los datos a un servidor donde guardan la información en una base de datos y a través de internet llegan a una aplicación utilizando The Things Network de la cual se hablará mas adelante en el marco teórico.

2.1.3. Implementación de LoRaWAN desde el dispositivo final hasta la aplicación

Este artículo [3] habla sobre la implementación de una red completa LoRaWAN. El sistema resultante ofrece una infraestructura completa para el desarrollo de aplicaciones y experimentos. La red consiste en tres componentes principales. La herramienta llamada Chirpstack que es de código libre y actúa como un servidor LoRaWAN corriendo en una computadora personal. La puerta de enlace LoRaWAN que es un dispositivo de red independiente y la referencia al dispositivo final que está construida con componentes baratos para los autores.

El proyecto cuenta con distintos esquemáticos ligados a través de referencias bibliográficas del dispositivo o nodo final para su fácil replicación. Así mismo la jerarquía utilizada para comunicar todos los dispositivos es la siguiente demostrada

en la Figura 2.4

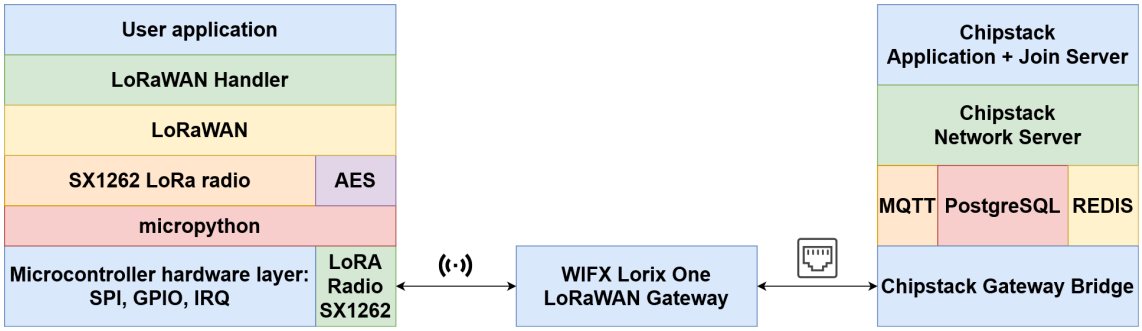


Figura 2.4: Jerarquía de componentes LoRa

Así mismo la solución propuesta por los autores esta implementada en MicroPython y se aprovecha de librerías existentes para el manejo de la tecnología y comunicación LoRaWAN.

El servidor de red LoRaWAN está corriendo en una computadora personal aprovechando las capacidades de Docker para la generación de toda la capa de herramientas necesarias para su funcionamiento.

Como resultados, los autores lograron obtener un servidor totalmente funcional corriendo en un contenedor que a su vez registra eventos ocurridos en el dispositivo final. Se hablan de las características pendientes por implementes, sin embargo, están satisfechos con el resultado debido a que la red es completamente funcional sin estas características.

2.1.4. Adquisición flexible de datos mediante LoRaWAN y MQTT para pequeñas y medianas empresas

Este artículo [4] propone una solución de LoRaWAN y Message Queuing Telemetry Transport (MQTT) para pequeñas y medianas empresas, presentando una solución generalizada para distintos tipos de problemas que evade altos costos de inversión y poder obtener datos en un periodo corto de tiempo además la solución está basada en herramientas de código abierto, obteniendo las ventajas de este tipo de herramientas.

El enfoque propuesto es el siguiente en la Figura 2.5



Figura 2.5: Arquitectura general propuesta

El artículo contribuye en proponer una solución flexible y con una infraestructura simple para la adquisición de datos a través de una red de sensores LoRaWAN y su visualización a través de una plataforma de Internet de las Cosas.

El protocolo MQTT funciona como intermediario entre LoRaWAN y la arquitectura del servidor asi como habilitador para la transferencia de datos de otras fuentes.

Asi mismo, este contribuye en aplicaciones practicas y actuales usando sensores de LoRaWAN asi como maneras de comunicarse a través de MQTT.

La solución propuesta viene de 3 componentes principales como se muestra en la Figura 2.6:

- Centro de datos
- Adquisición de datos con LoRaWAN
- Plataforma de Internet de las cosas

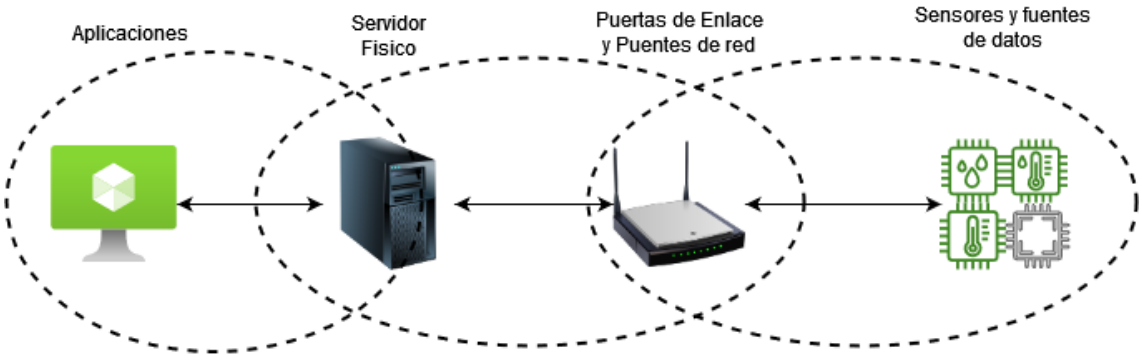


Figura 2.6: Arquitectura general propuesta

El protocolo intermediario MQTT es el centro de datos, diseñado para la conexión con dispositivos de recursos limitados. También este mismo habilita la comunicación entre sistemas utilizando el patrón de diseño de publicación y suscripción como se muestra en la Figura 2.7.

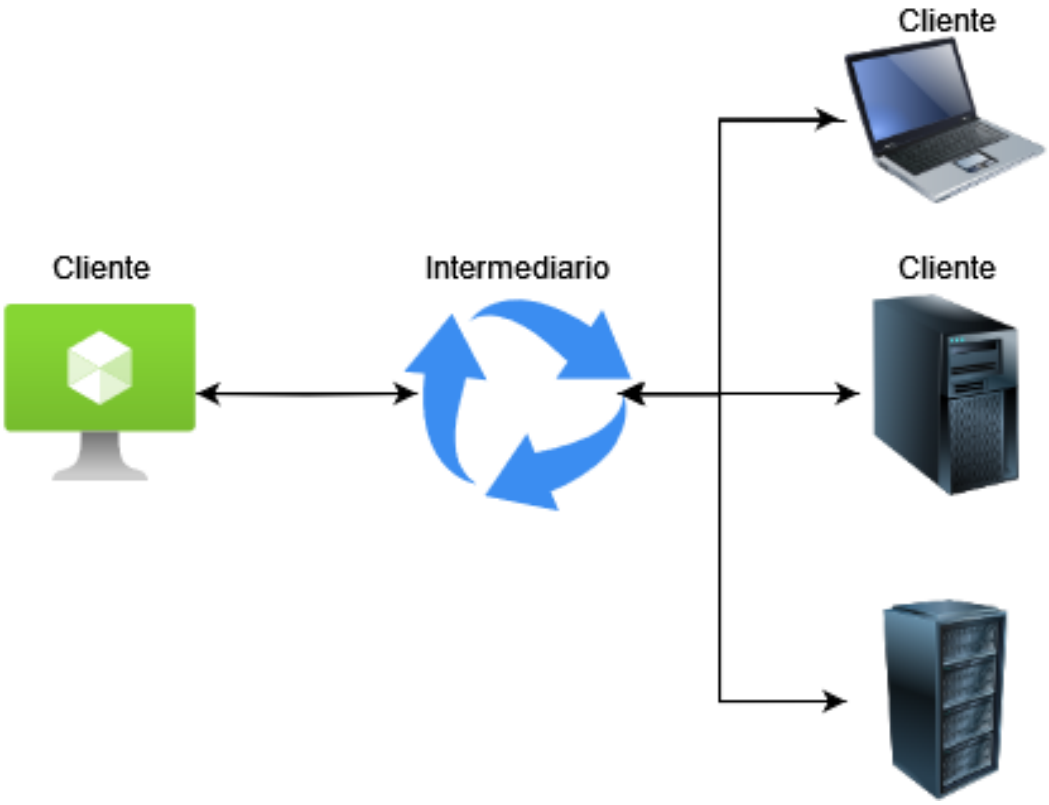


Figura 2.7: Arquitectura MQTT propuesta

El protocolo LoRaWAN es utilizado como una red inalámbrica de sensores, proporcionando una comunicación de larga distancia. Los sensores operados por batería son fácilmente desplegables en este tipo de protocolos y son integrados a través de una interfaz web, lo que hace que no sea necesario conocimientos de programación para su administración como se puede ver en la Figura 2.8.

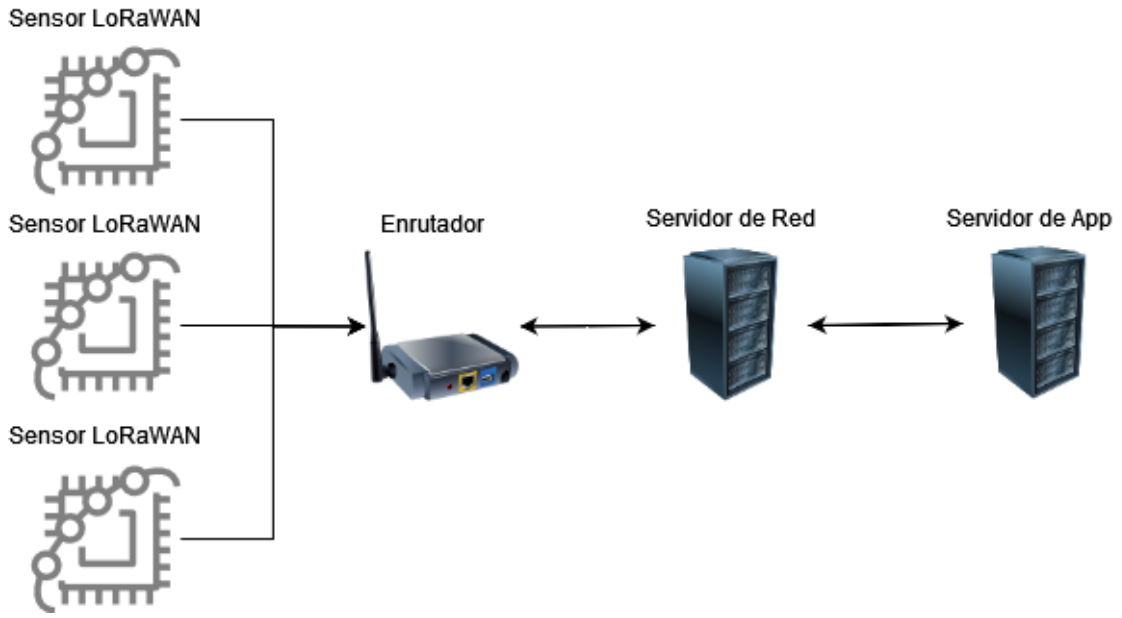


Figura 2.8: Arquitectura LoRa propuesta

La plataforma IoT es desplegada como se ve en la Figura 2.9 y utilizada para medir los datos obtenidos en un panel web. Esto propicia a una visualización y medición con poco esfuerzo, además de permitir agregar otras fuentes de datos gracias a su integración con MQTT



Figura 2.9: Plataforma IoT propuesta

El sistema implementado puede gestionar aproximadamente 5.000 mensajes MQTT por segundo con cargas útiles inferiores a 1.000 bytes, según la evaluación de rendimiento presentada por los autores. En conclusión, este trabajo demuestra el conjunto de herramientas en un solo proyecto con múltiples tipos de aplicación, el cual definitivamente es una buena base para el presente proyecto.

2.1.5. Diseño de una pasarela LoRaWAN con servidores autónomos para comunidades sin conexión a Internet

Este artículo [5] habla de como los dispositivos IoT en la práctica deberían ser sencillos de desplegar asi como deberían durar activos largos periodos en campo sin ninguna intervención. Se propone el uso del protocolo LoRaWAN para conseguir estos objetivos, además, se habla del uso de un conjunto de herramientas llamado The Things Networks el cual en Europa es fácilmente accesible ya que cuenta con dispositivos usables en campo para conectar con dispositivos LoRaWAN finales.

Debido a que no se puede garantizar siempre tener internet en muchos países, se propone una solución utilizando nodos IoT locales a una puerta de enlace LoRaWAN sin la necesidad de emplear redes de internet, a un bajo costo y poco poder de consumo.

Los investigadores implementaron tres bloques de la arquitectura LoRaWAN (Puerta de enlace, Servidor de red, Servidor de aplicación) en un solo dispositivo utilizando una Raspberry Pi y un modulo LoRaWAN. El acceso local a los datos de la aplicación es a través de un punto de acceso WiFi, permitiendo a los miembros de la comunidad acceder a sus datos comunitarios incluso cuando no hay internet. Además de datos IoT en tiempo real, los investigadores proponen servicios extra que pueden ser implementados en el repositorio para ayudar en el desarrollo de la educación, salud y otros contenidos que pueden apoyar a comunidades distantes.

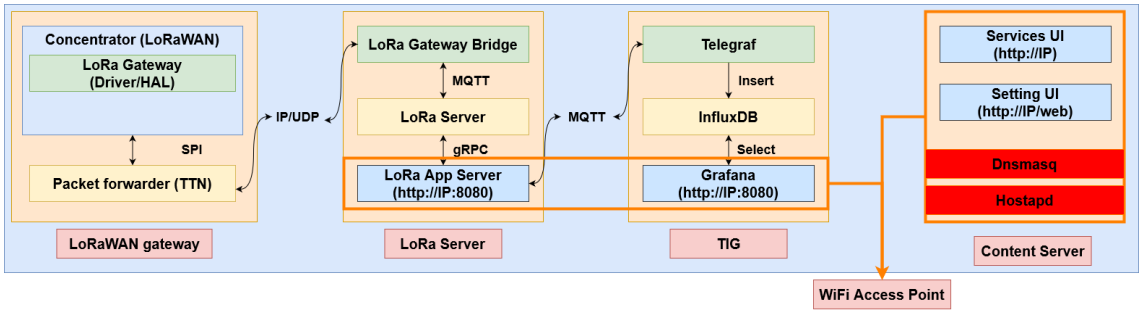


Figura 2.10: Partes de una Estación Base

El dispositivo completo con todas estas características es llamado la Estación Base (BS en inglés). La arquitectura de la estación base es dividida en cuatro partes como se muestra en la Figura 2.10: Puerta de enlace LoRaWAN (GW en inglés), Servidor de aplicación y de red, un componente llamado TIG (De las siglas Telegraf, InfluxDB y Grafana), el punto de acceso WiFi y el servidor de contenido

Puerta de enlace LoRaWAN Figure 2.11. Actúa como un transmisor de paquetes entre un Nodo final y un Servidor de red basado en la implementación de código abierto propuesta por Semtech y construida en una Raspberry Pi. El equipo de investigadores cuenta con dos versiones, la más robusta que usa un módulo SX1301 la cual admite comunicación simultanea mientras escucha en ocho diferentes frecuencias y la mas barata utilizando un módulo RFM96W el cual solo admite un solo canal.

Servidor de red y servidor de aplicación. El rol del servidor de red es autenticar nodos y estar seguro de que solo se reciben datos de los dispositivos registrados. El servidor de aplicación provee procesamiento de tramas y su desencriptación. Asi mismo, ofrece al usuario final los datos solicitados, como el clima, la calidad del aire, alarmas, etc. Los datos son localmente procesados, guardados y se pueden accesar a través del punto de entrada WiFi

TIG. La herramienta de Telegraf es usada para suscribirse a un intermediario de mensajes MQTT y recolectar mensajes en formato JavaScript Object Notation (JSON) los cuales son transmitidos por el servidor de aplicación LoRa y esto se inyectan en InfluxDB una base de datos de series de tiempo (TSDB en inglés). Gracias a Grafana (Una plataforma de código abierto para gráfico de datos) las mediciones pueden ser obtenidas de InfluxDB para su análisis y monitoreo.

Servidor de Contenido. El servidor de contenido provee acceso al contenido estático, como el material educacional, información de salud, las mejores prácticas de agricultura, etc. Asi como un tablero que funciona como boletín de información para la comunidad. El punto de acceso WiFi hace este material disponible a través de la comunidad usando una interfaz web.

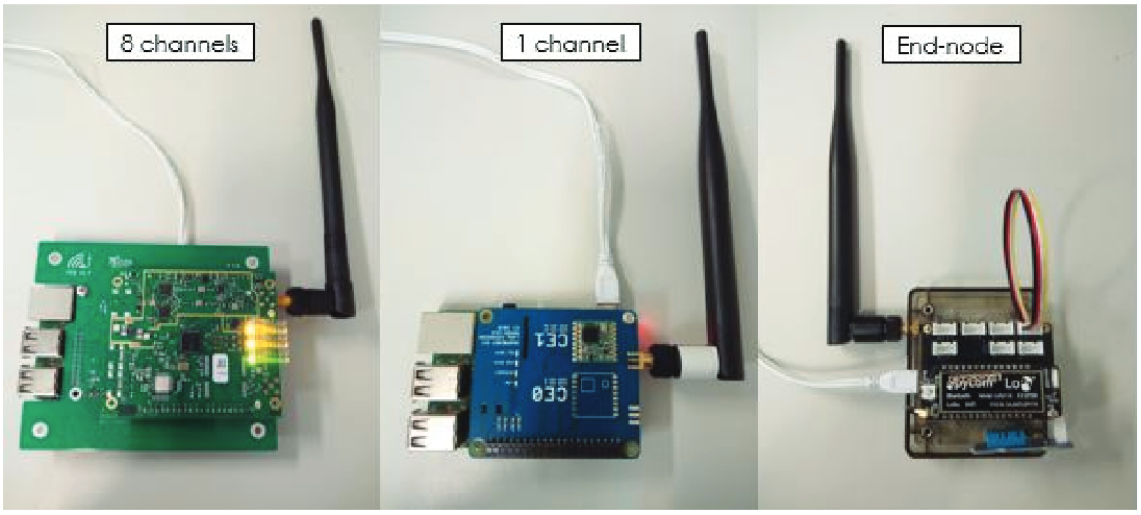


Figura 2.11: BS con 8 canales (izquierda), 1 canal (centro) y un nodo final (derecha).

En conclusión, este trabajo está enfocado a extender los beneficios de utilizar IoT en personas que viven en áreas con muy limitado o sin acceso a conexiones de internet, utilizando redes y servidores LoRaWAN los cuales pueden ser encapsulados en dispositivos pequeños que proveen todas las funcionalidades requeridas. Asi mismo pueden tomar ventaja de dispositivos TTN si es que están disponibles, de forma intermitente, incluso pueden ser soluciones independientes que sirvan a una comunidad. Los usuarios pueden interactuar con el BS propuesto utilizando sus dispositivos propios a través de la conexión WiFi. El trabajo futuro de los investigadores se centrará en el tablón de anuncios y el contenido sistemas de gestión que son características muy útiles para comunidades aisladas, así como en una extensión de este modelo al acomodar la sincronización de la base de datos de varias BS desplegadas en la misma comunidad, interconectadas entre sí pero independientes de una conexión a Internet.

2.1.6. Particionado de red de acceso radio en redes LoRaWAN

En el siguiente trabajo de final de grado [6] vemos que en este se propone dividir los recursos de una red LoRaWAN en segmentos o *slices*, que puedan asignarse a distintos usuarios o proveedores. Para ello, define un bloque de recursos (RB) como la combinación de un canal y un factor de dispersión (SF). En la configuración estudiada, los ocho canales de 125 kHz y los SF del 7 al 12 permiten formar hasta

48 combinaciones de recursos.

La solución se implementa sobre una red LoRaWAN con ChirpStack y dispositivos reales. El servidor asocia cada dispositivo con un *slice* y, durante el intercambio de mensajes de unión, envía al dispositivo una máscara que indica los RB disponibles. A partir de esa máscara, el dispositivo selecciona un RB y configura el canal y el SF de transmisión según las condiciones de la señal, estimadas mediante indicador de fuerza de señal recibida (RSSI) o la relación señal-ruido (SNR). Así, la selección del SF puede ajustarse a la calidad del enlace, mientras que la separación por canal y SF permite organizar las transmisiones de distintos usuarios.

La arquitectura utilizada para implementar esta solución se resume en la Figura 2.12. Las micro tarjetas o como el autor se refiere motas de modelo LoPy4 se comunican con la pasarela mediante LoRaWAN, y esta reenvía los paquetes al servidor, donde ChirpStack y el sistema de particionado gestionan la asignación de recursos.

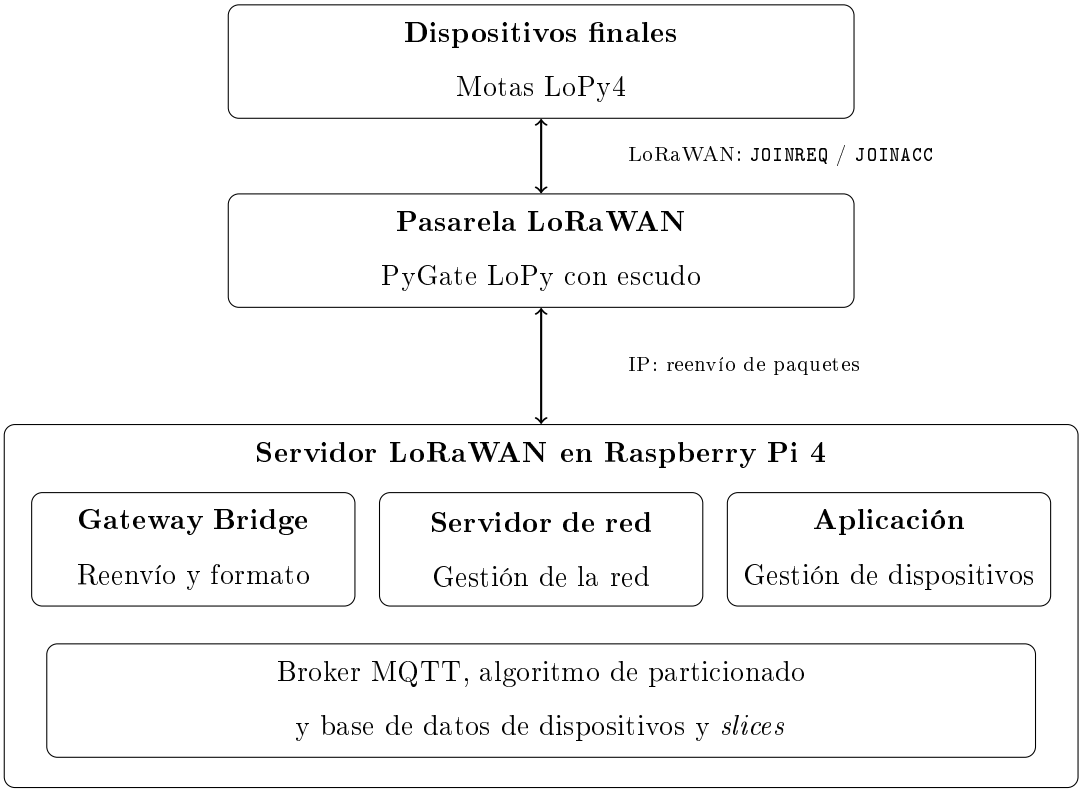


Figura 2.12: Arquitectura utilizada para el particionado de la red LoRaWAN

Las pruebas confirmaron la comunicación entre los dispositivos y el servidor, así como la asignación de recursos según el SNR. En particular, las simulaciones mostraron que, para los valores de SNR evaluados, el algoritmo no asignaba factores de dispersión incompatibles con la calidad de la señal. En cambio, no fue posible

validar el algoritmo basado en RSSI, pues durante las pruebas no se alcanzaron condiciones de cobertura suficientes para que funcionara. El trabajo concluye que el particionado propuesto es viable en LoRaWAN y plantea como líneas futuras ampliar los criterios de selección, analizar el rendimiento y el consumo energético, y permitir que el usuario elija el *slice* según los requisitos de calidad de servicio.

2.1.7. Diseño de un Nodo LoRa-GPS para Localización de Bicicletas Implementado en una Red LoRaWAN

El trabajo de fin de grado de Jairo Andrés Rodríguez Sánchez y Jhoiner Smith Rojas González [7] aborda el diseño de un nodo para localizar bicicletas mediante la integración de posicionamiento GPS y comunicación LoRaWAN. A nivel funcional, el nodo obtiene las coordenadas de la bicicleta y las transmite a través de la red para que puedan ser recibidas y consultadas por el usuario. Este enfoque aprovecha el alcance de LoRaWAN para comunicar un dispositivo móvil con una aplicación de seguimiento.

La Figura 2.13 presenta un esquema general del flujo de información: desde la obtención de la posición en la bicicleta hasta su consulta en la aplicación. Se muestran los componentes funcionales de una arquitectura LoRaWAN, sin especificar modelos de hardware ni plataformas de software particulares.

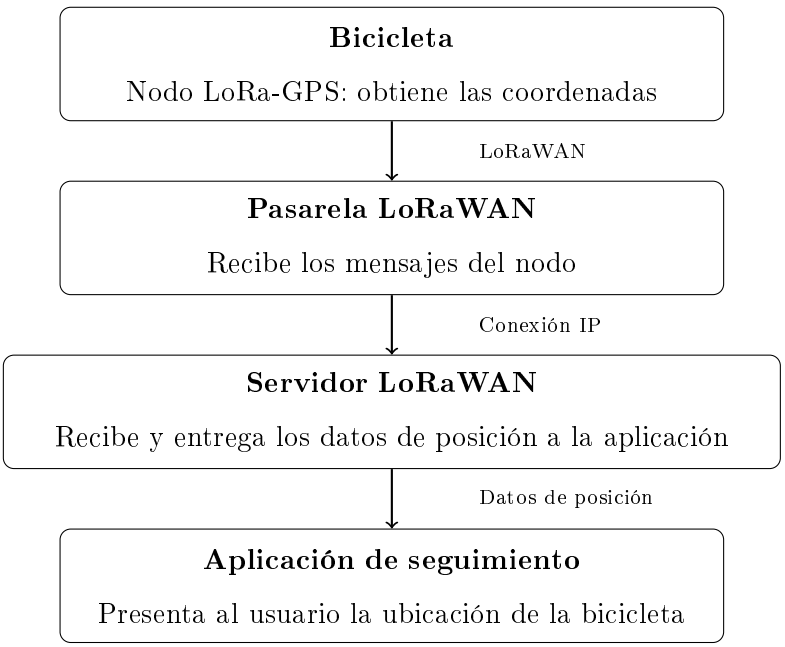


Figura 2.13: Esquema funcional general de localización de bicicletas con LoRaWAN

2.2. Marco teórico

2.2.1. LoRa y LoRaWAN

LoRa es una especificación de una red LPWAN propuesta por LoRa Alliance [8] y pensada para comunicar dispositivos de bajo costo y bajo consumo de energía alimentados por baterías. La especificación cubre la capa física y la de acceso al medio de la red, dejando a las aplicaciones el resto de las capas. La comunicación entre los dispositivos y las puertas de enlace (nombradas gateways por la especificación en inglés) se basa en la modulación LoRa desarrollada por la compañía Semtech [9]. Esta modulación permite enlaces de varios kilómetros inclusive en entornos urbanos, gracias a su presupuesto de enlace máximo de 155 dB, con una tasa de bit entre 0.3 kbps y 50 kbps. En cuanto a las particularidades de la modulación LoRa, diferentes dispositivos emitiendo en la misma frecuencia, pero con tasa de bit distinta, no corrompen las transmisiones. Esto permite disponer de un amplio número de canales virtuales para la comunicación entre dispositivos finales y estación base. En la siguiente Figura 2.14 se muestra la pila de protocolos que utiliza LoRa.

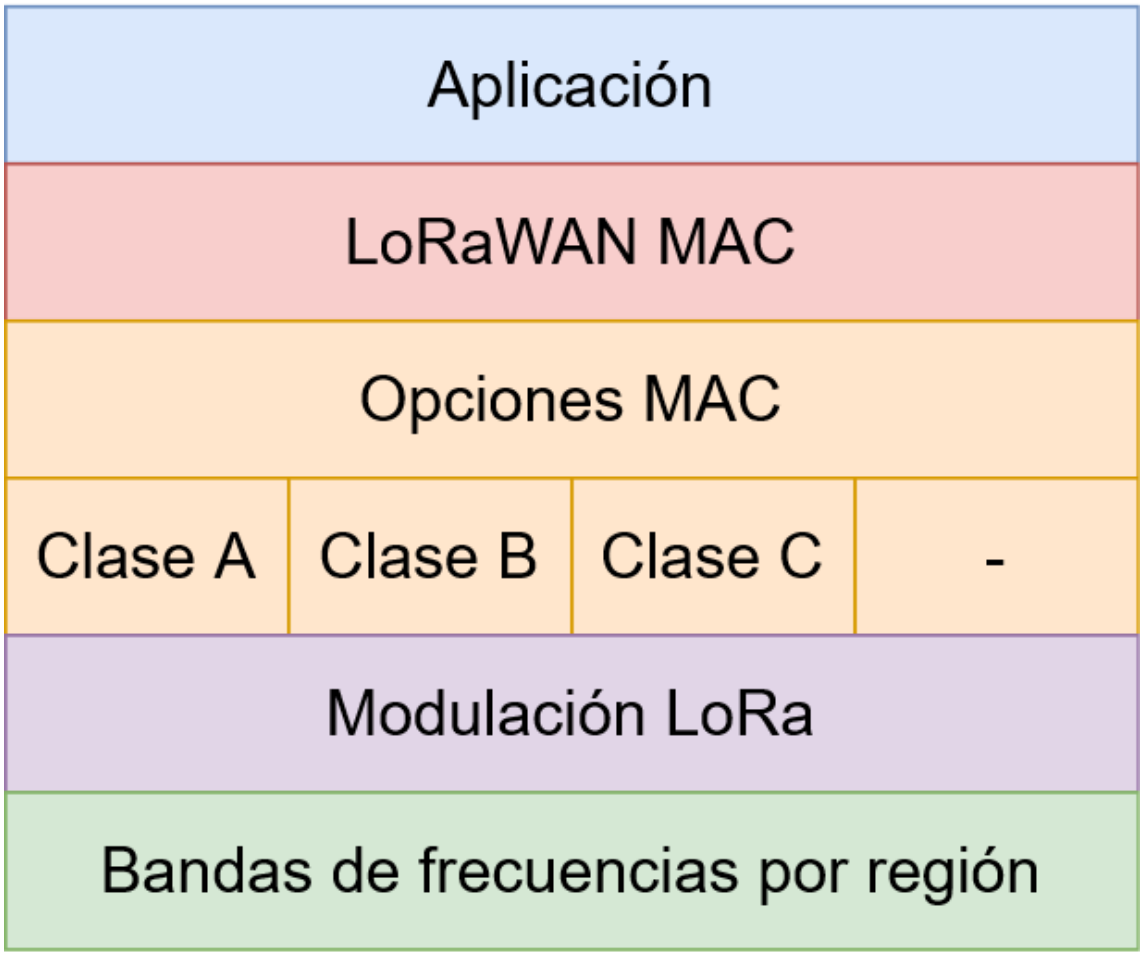


Figura 2.14: Arquitectura LoRa

La topología de red propuesta por LoRaWAN es estrella, es decir, los dispositivos finales se comunican directamente a un salto con una estación base de forma bidireccional. Las estaciones base transmiten los datos de los dispositivos finales al servidor de red y viceversa utilizando una conexión IP estándar. En la Figura 2.15 se muestra la arquitectura utilizada en redes LoRa.

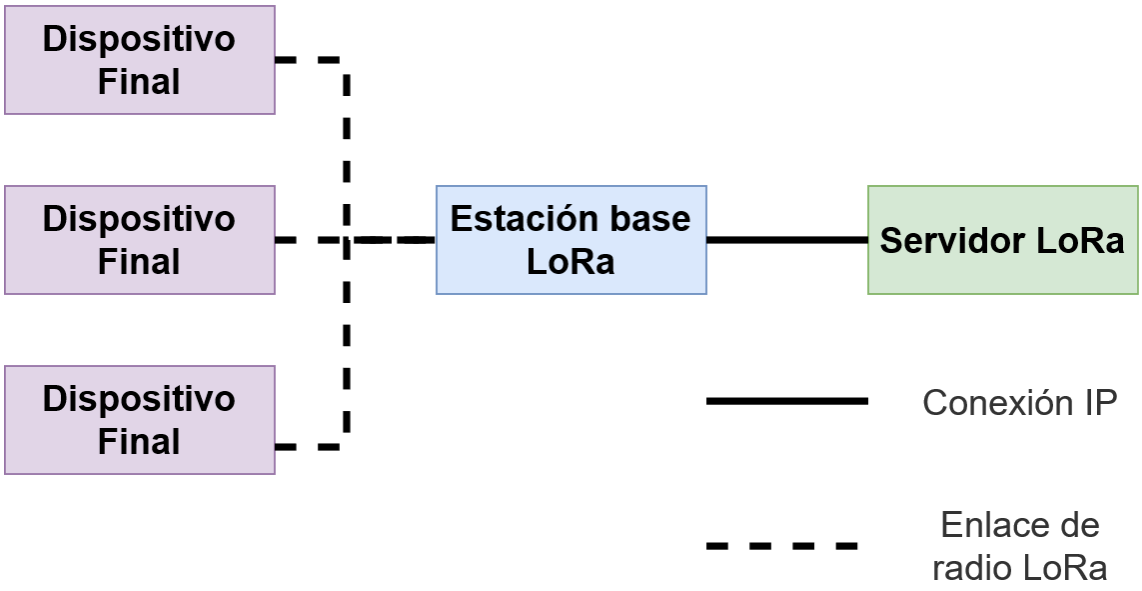


Figura 2.15: Topología LoRa

2.2.2. Docker

Docker es un motor de contenedores de código libre que utiliza una virtualización [10] ligera permitiendo automatizar el despliegue de aplicaciones mediante Contenedor virtuales. Esto significa que puede encapsular las aplicaciones dentro de un contenedor junto con sus dependencias y ejecutarlos sobre cualquier equipo que tenga instalada la plataforma Docker, usualmente servidores Linux, nubes públicas y privadas. Docker amplía LXC con un kernel y una interfaz de programación de aplicaciones (API) a nivel de aplicación que permiten la ejecución de procesos aislando CPU, memoria, E/S, red, etc. Además, Docker utiliza diferentes espacios de nombre para aislar por completo la vista de una aplicación del entorno subyacente (sistema operativo), lo que incluye los árboles de procesos, la red, los IDs de usuario y los sistemas de archivos

2.2.3. The Things Network

The Things Network (TTN), es una red de IoT global, descentralizada, gratuita, abierta y en la que cualquier persona puede aportar sus conocimientos acerca de IoT con la finalidad de mejorar cada vez más la plataforma. TTN fue creado en el verano de 2015 en Amsterdam, por sus dos fundadores Wienke Giezeman y Johan Stokking [11], entre otros y cuyo objetivo era facilitar el diseño y la creación de redes, para lo cual daba soporte a las distintas puertas de enlace distribuidos por el mundo y además ofrecía la infraestructura de back-end a sus usuarios, es decir, una implementación de las funcionalidades del servidor de red y del servidor de aplicación.

Este set de herramientas del lado del servidor se enfrenta a duplicidades de mensajes, gestiona la transmisión de mensajes descendentes, gestiona integraciones con plataformas, como HTTP o MQTT y ofrece una serie de APIs en diferentes lenguajes de programación como Java, Node.js o Node-Red.

Como se trata de una plataforma código abierto, los códigos con los que se ha construido el set de herramientas del lado del servidor son públicos y se puede acceder a ellos a través de un proyecto en GitHub. Actualmente TTN tiene registradas alrededor de 9000 puertas de enlace distribuidos por todo el mundo y están registrados más de 80000 miembros.

Teniendo en cuenta que hace tan solo un año había registradas 3000 puertas de enlace, se puede pensar que la idea de la evolución tan rápida que está teniendo la tecnología LoRaWAN y a su vez la red de TTN. En la siguiente Figura 2.16 se puede observar la distribución de las puertas de enlace en el mundo: Este continuo crecimiento de la comunidad permite a muchas personas acceder de manera muy sencilla y gratuita a un entorno IoT donde poder realizar proyectos o implementar sus ideas.



Figura 2.16: Mapa con la distribución de las puertas de enlace en el mundo para The Things Network

Además, TTN ha desarrollado una gran cantidad de comunidades en las que los usuarios, generalmente ubicados en las mismas zonas geográficas puedan intercambiar información o conocimientos que puedan ser útiles para otros usuarios y así, de manera recíproca pueden servirse de ayuda unos con otros. Por ejemplo, en España se han creado recientemente varias comunidades de TTN en Mexico, Estados Unidos, Países en Suramérica, etc., lo cual indica que se está notando el auge de LoRaWAN de forma más cercana.

Capítulo 3

Metodología

3.1. Diseño del servidor LoRa

Descripción

Como se explicó en el capitulo anterior es necesario un conjunto de tecnologías distintas para lograr nuestro propósito de generar un servidor que administre los distintos nodos. En la siguiente sección se explica con mas detalle toda la arquitectura propuesta para la implementación del servidor maestro.

El objetivo es desarrollar todo un ecosistema IoT completo que sea lo suficientemente genérico para poder ser soportado y redesplegado en distintos proyectos, pero al mismo tiempo lo suficientemente especializado para cumplir las funciones de administrar los nodos asi como su manejo de la información.

3.2. Diseño de arquitectura implementado

Diagrama de arquitectura

Tras la investigación del estado del arte decidí implementar la siguiente arquitectura de configuración basándome en el estado del arte de proyectos anteriores. Por lo que encontraran bastante similitud con los mismos, aunque por supuesto con diferencias específicas para apoyar al equipo de investigadores que usaran el sistema. En la Figura 3.1 se muestra el diagrama final.

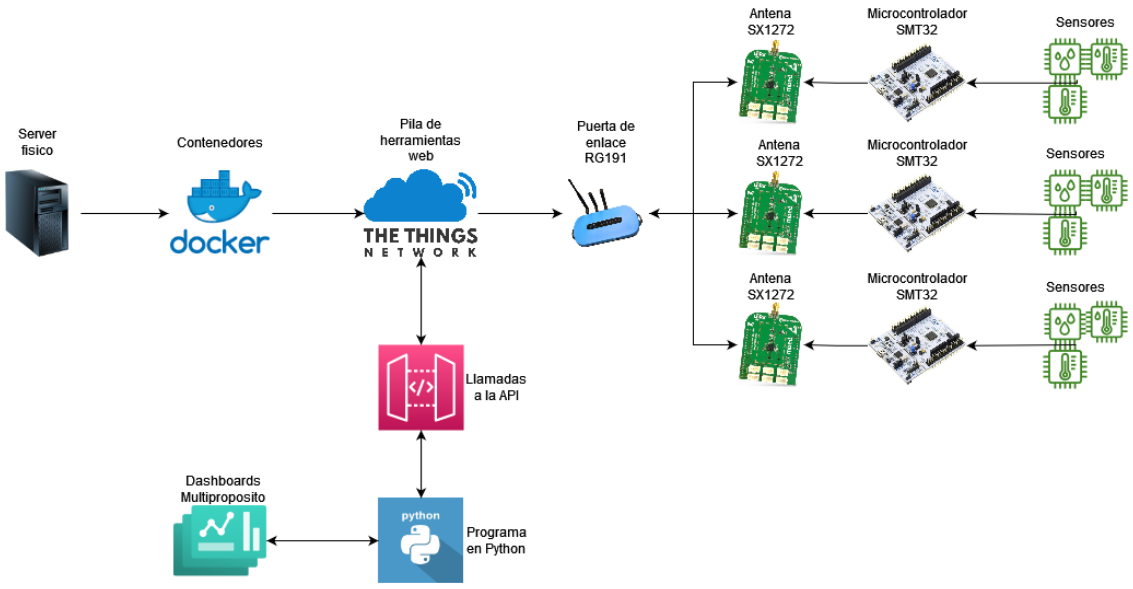


Figura 3.1: Arquitectura implementada para el sistema de nodos

El servidor físico cuenta con una instancia de Docker para facilitar su portabilidad a otros sistemas distintos o inclusive su montaje en la nube. Dentro del contenedor se encuentra instalado The Things Stack el cual nos facilita la comunicación con múltiples puertas de enlace de tipo RG191 asi como con la administración de los nodos LoRa que funcionan atraves del microcontrolador SMT32 con su antena SX1272. A continuación, explicare todos los pasos necesarios para la configuración y lanzamiento de un servidor.

3.3. Configuración del servidor LoRa

3.3.1. Prerrequisitos

Primeramente, debemos tener configurado nuestro entorno donde todo el sistema vivirá. Para este caso de uso utilizamos una distribución Debian. Para instalarlo podemos utilizar la guía oficial proporcionada por Debian, sin embargo, tal y como explique antes. El sistema puede ser montado en cualquier sistema operativo o inclusive en la nube.

3.3.2. Instalación de Docker

Una vez que tengamos una distribución de Linux en funcionamiento, debemos instalar Docker.Para mayor comodidad Debian ya proporciona una guía de como

instalar Docker de forma sencilla en su distribución

Una vez instalado Docker podemos probar la instalación:

```
sudo docker run hello-world
```

Ahora, con docker instalado, podemos instalar docker-compose para facilitar la gestión de diferentes contenedores.

La forma más fácil de instalar es verificar la última versión aquí y descargar los archivos binarios más recientes usando el comando curl:

```
sudo curl -L https://github.com/docker/compose/releases/
download/1.25.3/docker-compose-`uname -s`-`uname -m`
-o /usr/local/bin/docker-compose
```

A continuación, debemos configurar los permisos.

```
sudo chmod +x /usr/local/bin/docker-compose
```

Finalmente, solo verifique que esté funcionando

```
docker-compose --version
```

3.3.3. Instalación de The Things Stack

El siguiente paso para crear una instalación limpia es descargar los archivos de The Things Stack la cual como se explico anteriormente es la suite utilizada para el manejo de los dispositivos LoRa. Lo primero que debemos hacer es crear una estructura de carpetas con los siguientes comandos:

```
mkdir the-things-stack
mkdir config
mkdir config/stack
```

Descargar los archivos necesarios y cámbiarle el nombre para que se pueda utilizar en Docker

```
curl https://www.thethingsindustries.com/docs/
getting-started/installation/configuration/
docker-compose-open-source.yml > docker-compose.yml
```

```
curl https://www.thethingsindustries.com/docs/
getting-started/installation/configuration/
ttn-lw-stack-docker-open-source.yml > ttn-lw-stack-docker.yml
```

Modifique los archivos para satisfacer sus necesidades. Recuerde que puede usar archivos en el repositorio como ejemplo o seguir la guía de configuración oficial.

Descargar las imágenes de dockerfile

```
sudo docker-compose pull
```

Ahora tenemos las imágenes listas para usar.

3.3.4. Configuración de certificados

Antes de que podamos continuar configurando las imágenes de Docker, debemos asegurarnos de que podremos conectarnos a nuestro panel web sin problemas, para hacer esto, necesitamos configurar algunos certificados.

En el repositorio ya están los certificados necesarios para realizar esta tarea, en caso de que quieras crear tus certificados desde cero puedes seguir la guía de configuración oficial

Para generar certificados, necesitaremos herramientas cfssl. Esta herramienta está construida con el lenguaje de programación Go, por lo que, para empezar, debemos descargar la versión más reciente de la página web de Go.

Antes de que podamos continuar configurando las imágenes de nuestra ventana acoplable, debemos asegurarnos de que podremos conectarnos a nuestro tablero sin problemas, para hacer esto, necesitamos configurar algunos certificados.

En el repositorio ya están los certificados necesarios para realizar esta tarea, en caso de que quieras crear tus certificados desde cero puedes seguir la guía de configuración oficial

Despues:

```
#Descomprimir Go
tar -xzf go1.19.4.linux-amd64.tar.gz


#Mover GO a la carpeta local
sudo mv go /usr/local/
```

```
#Configurar GOROOT env var
export GOROOT=/usr/local/go

#Verificar instalacion
go --version
```

Ahora, con Go ejecutándose, necesitamos descargar las herramientas de cfssl. Podemos hacerlo desde el repositorio de cfssl.

```
git clone https://github.com/cloudflare/cfssl.git

#Moverse dentro de la carpeta generada
cd cfssl

#Construir el repositorio
make
```

Una vez que los pasos sean exitosos, se creará una carpeta bin, agregue esta ruta a la variable de entorno PATH como esta

```
export PATH="/home/uaguser/Downloads/cfssl/bin:$PATH"
```

Ahora necesitamos crear el archivo de certificado llamado ca.json con el siguiente contenido

```
{
    "names": [
        {
            "C": "NL",
            "ST": "Noord-Holland",
            "L": "Amsterdam",
            "O": "The Things Demo"
        }
    ]
}
```

Este es un ejemplo de los datos que puede modificar a su gusto siguiendo las reglas del certificado Luego ejecute lo siguiente para generar la clave y el certificado de CA

```
cfssl gencert -ca ca.pem -ca-key ca-key.pem cert.json | cfssljso
```

Ahora escriba la configuración de su certificado en cert.json

```
{
    "hosts": ["thethings.example.com"],
    "names": [
        {
            "C": "NL",
            "ST": "Noord-Holland",
            "L": "Amsterdam",
            "O": "The Things Demo"}
    ]
}
```

Ejecute el siguiente comando para generar la clave del servidor y el certificado

```
cfssl gencert -ca ca.pem -ca-key ca-key.pem
cert.json | cfssljson -bare cert
```

Ahora cambie el nombre de su certificado cert-key.pem a key.pem

```
mv cert-key.pem key.pem
```

Debian solo reconoce certificados en formato crt, por lo que es hora de convertir ambos certificados

```
openssl x509 -outform der -in ca.pem -out ca.crt
openssl x509 -outform der -in cert.pem -out cert.crt
```

Copie su certificado en la carpeta de certificados, esto puede ser diferente según su distribución de Linux

```
sudo cp ca.crt /usr/local/share/ca-certificates/
```

Cambie los permisos de sus certificados

```
sudo chmod 600 /usr/local/share/ca-certificates/ca.*
```

```
chmod 777 ca.pem
chmod 777 cert.pem
chmod 777 key.pem
```

Actualice sus certificados

```
sudo update-ca-certificates
```

Su árbol de archivos ahora debería verse así, excluyendo la carpeta blob

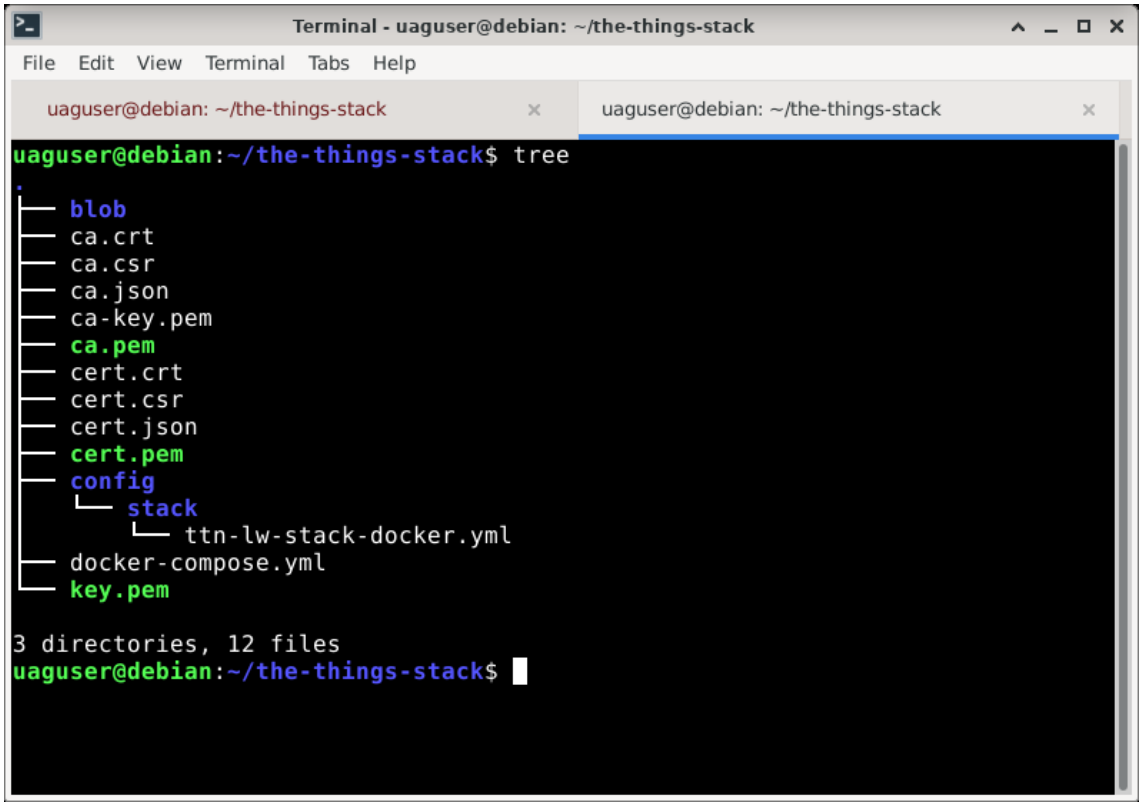


Figura 3.2: Árbol de archivos

3.3.5. Configurar LAIRD Sentrius RG191

Ahora que configuramos los certificados, estamos listos para configurar nuestro Laird Gateway y las IP del servidor para comunicarnos.

Primero, debemos acceder a la puerta de enlace de Laird para hacerlo, consulte el capítulo 4 de la documentación oficial *Inicie sesión en la puerta de enlace.*

Deberíamos ver algo como en la Figura 3.3.

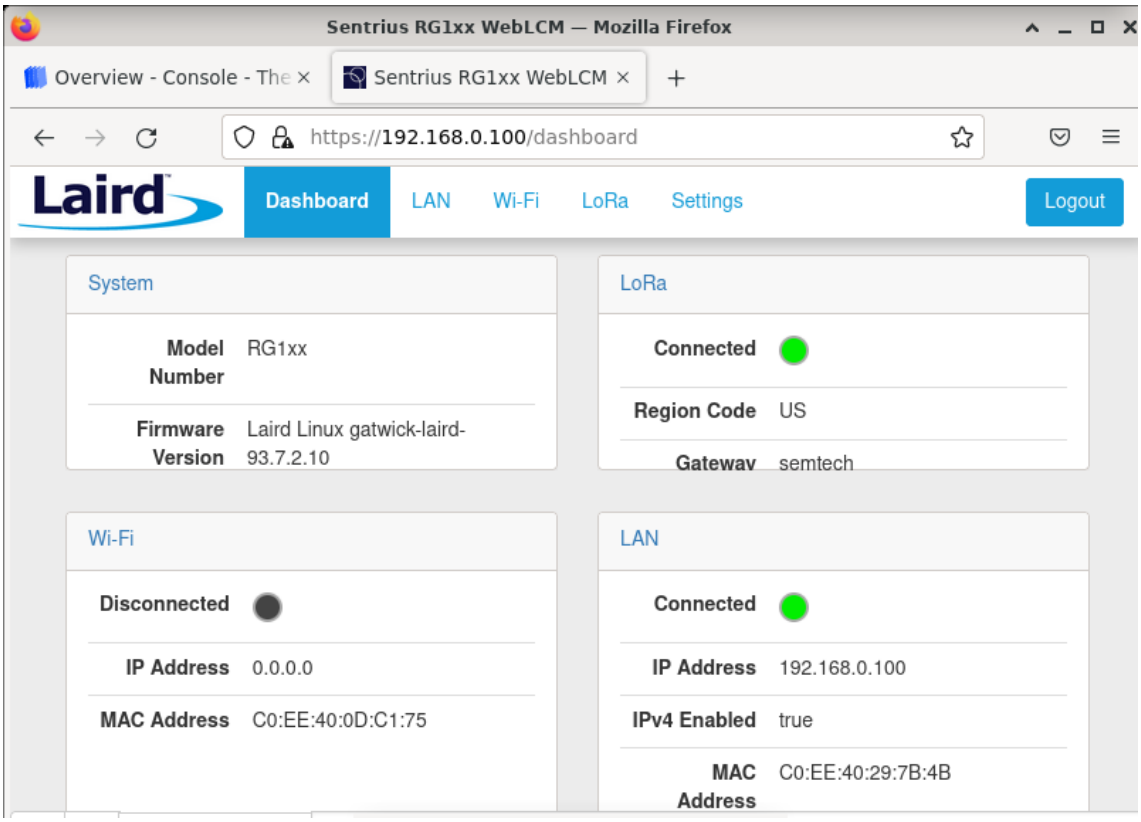


Figura 3.3: Página principal del RG191

Haga clic en la pestaña LAN y rellénelo con la siguiente información y guárdelo (Figura 3.4).

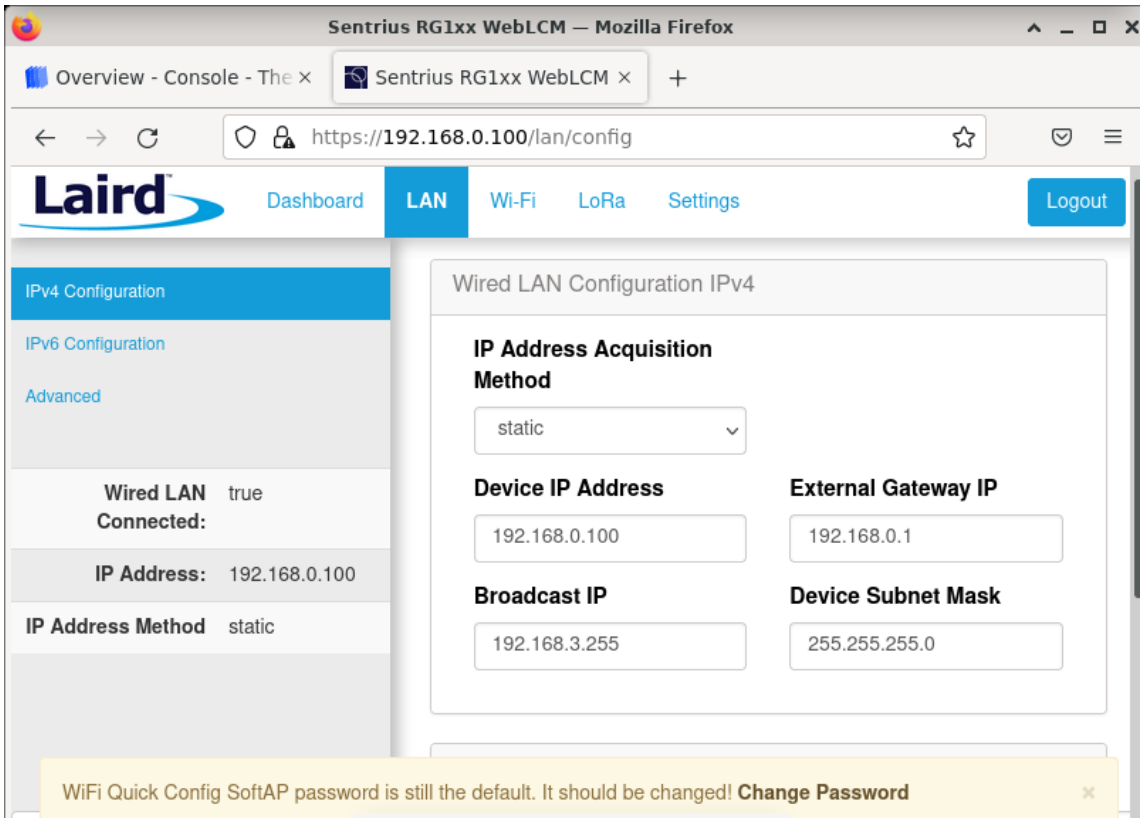


Figura 3.4: Modificación de IP Estática en el RG191

Luego haga clic en LoRa y luego en la sección Forwarder y complete con la siguiente información y guárdela (Figura 3.5).

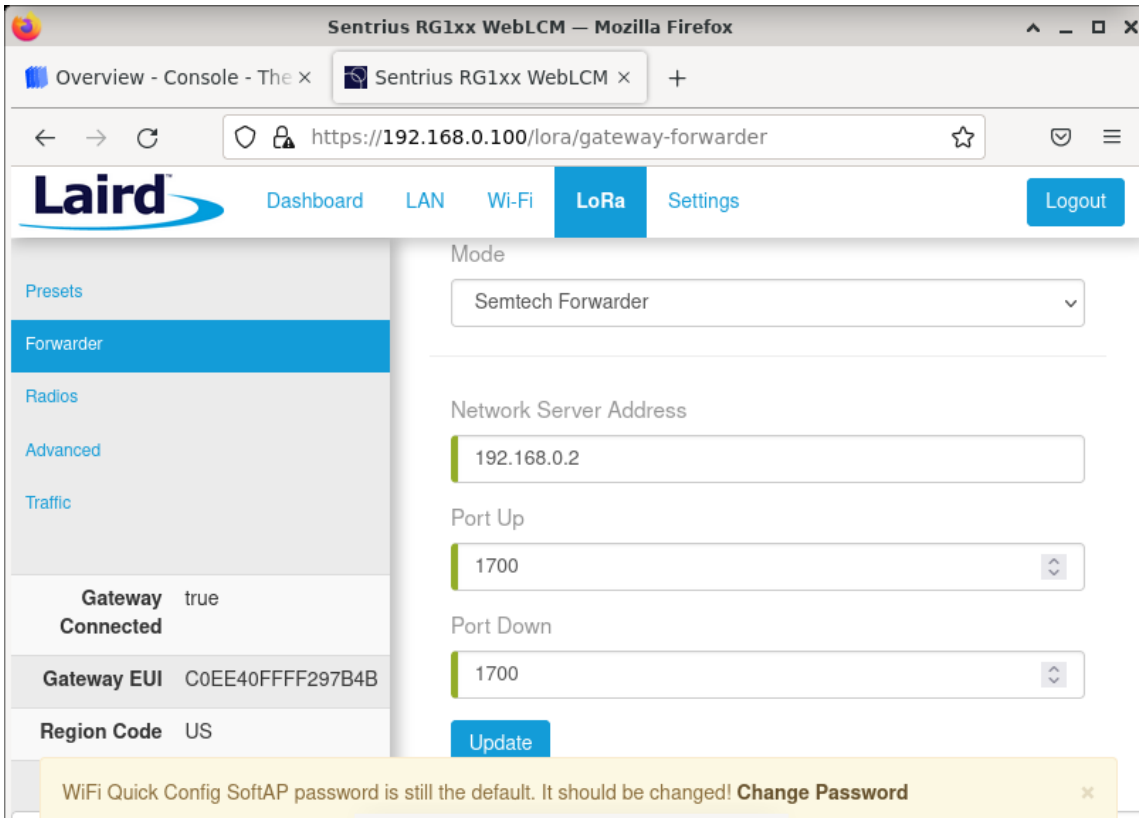


Figura 3.5: Modificación del protocolo LoRa en el RG191

Ahora regrese al servidor, debemos verificar las interfaces disponibles para usar para comunicarnos con la puerta de enlace para hacerlo, usamos el siguiente comando

```
ip link show
```

En nuestro caso vamos a usar enp3s0 como interfaz para comunicarnos con la puerta de enlace para modificarla use el siguiente comando

```
sudo vim /etc/network/interfaces
```

Agregue la siguiente información al final del archivo y guárdelo (Figura 3.6).

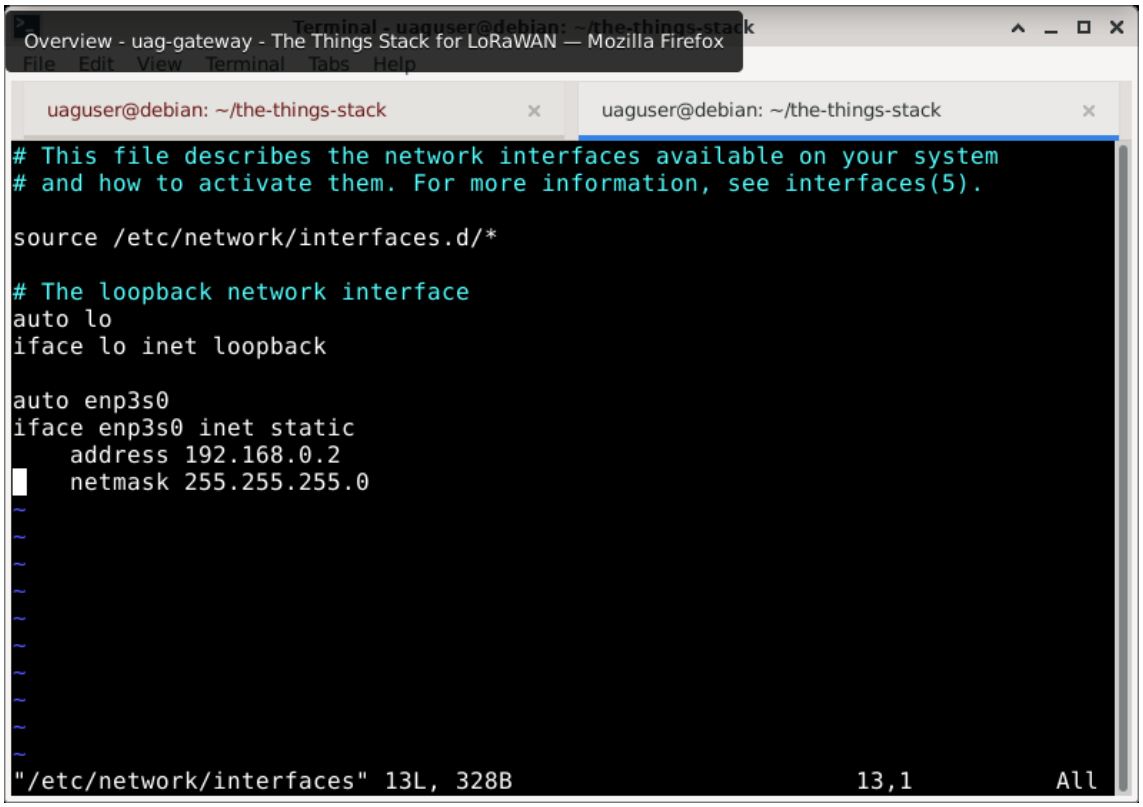


Figura 3.6: Interfaz enp3s0

Reinicie interfaces de red

```
sudo systemctl restart networking.service
```

Verifique que los cambios se aplicaron en la interfaz de red, en nuestro caso usamos enp3s0

```
ip -f inet addr show enp3s0
```

Verifique la conectividad entre Gateway y Server con el siguiente comando

```
ping 192.168.0.100
```

Si el ping no es exitoso, verifique nuevamente si la puerta de enlace y la interfaz están configuradas correctamente. Nota: debe perder el acceso a Internet después de este punto si usa IP privadas debido al cambio de IP de la interfaz.

3.3.6. Configurar imágenes de Docker

Una vez que la puerta de enlace y el servidor de Laird estén configurados con direcciones IP estáticas, ahora estamos listos para configurar nuestras imágenes de Docker

Primero necesitamos migrar la base de datos.

```
sudo docker-compose run --rm stack is-db migrate
```

Ahora cree y administre el usuario para el panel web

```
sudo docker-compose run --rm stack is-db create-admin-user
--id admin --email yourmail@gmail.com
```

Luego habilite oauth en el panel web

```
sudo docker-compose run --rm stack is-db create-oauth-client
--id cli     --name "Command Line Interface"     --owner admin
--no-secret     --redirect-uri "local-callback"
```

Cree estas dos variables de entorno para la contraseña de la consola y para la dirección del servidor

```
CONSOLE_SECRET="console"
SERVER_ADDRESS="192.168.0.2"
```

Generar usuario autenticado

```
sudo docker-compose run --rm stack is-db create-oauth-client
--id console --name "Console" --owner admin --secret
"${CONSOLE_SECRET}" --redirect-uri "${SERVER_ADDRESS}/console/
oauth/callback" --redirect-uri "/console/oauth/callback"
--logout-redirect-uri "${SERVER_ADDRESS}/console"
--logout-redirect-uri "/console"
```

Ahora estamos listos para ejecutar todos los contenedores.

```
sudo docker-compose up
```

Si todo es exitoso, deberíamos poder acceder al tablero en el siguiente enlace

```
https://192.168.0.2/console/
```

Ahora es el momento de pasar a configurar los dispositivos en el panale web

3.4. Configurar panel web

Una vez que accedemos al tablero desde el navegador, la primera página que deberíamos ver se verá así (Figura 3.7).

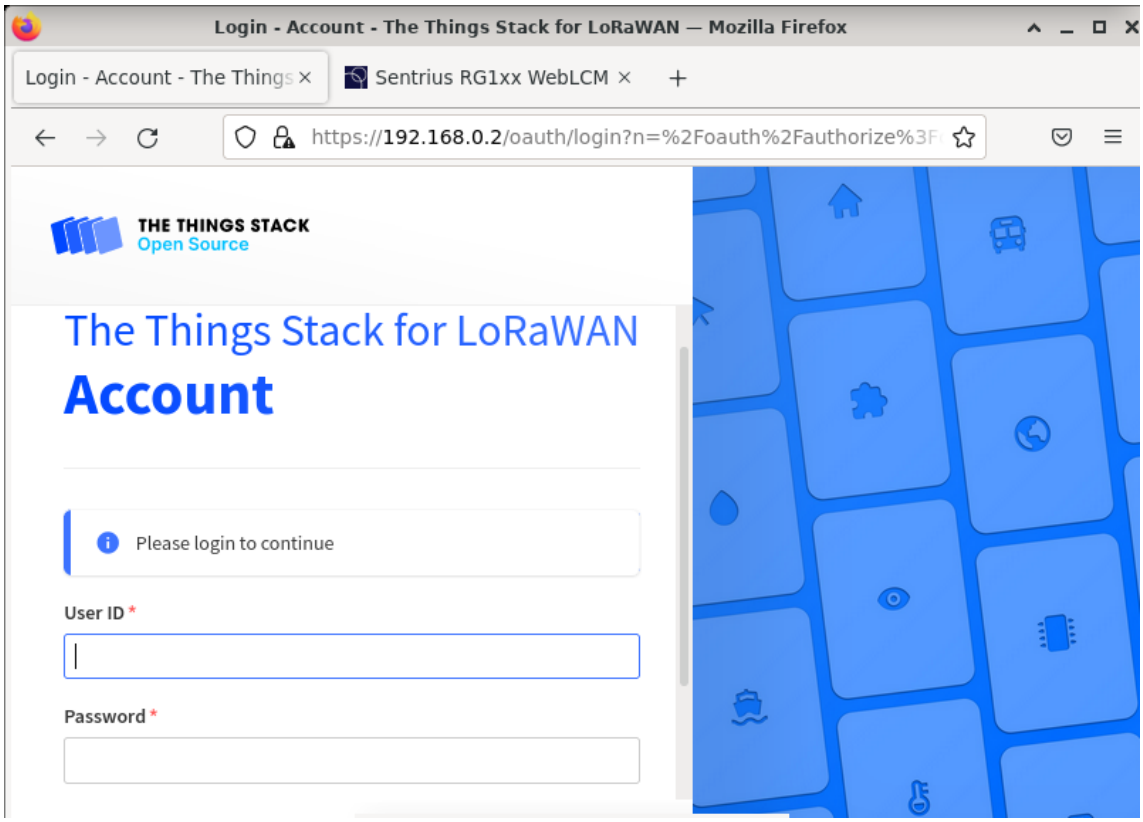


Figura 3.7: Página principal de The Things Stack

Use sus credenciales de administrador que generó previamente para iniciar sesión

3.4.1. Registrar puerta de enlace

Ahora, en la consola principal, haga clic en ir a las puertas de enlace (Figura 3.8).

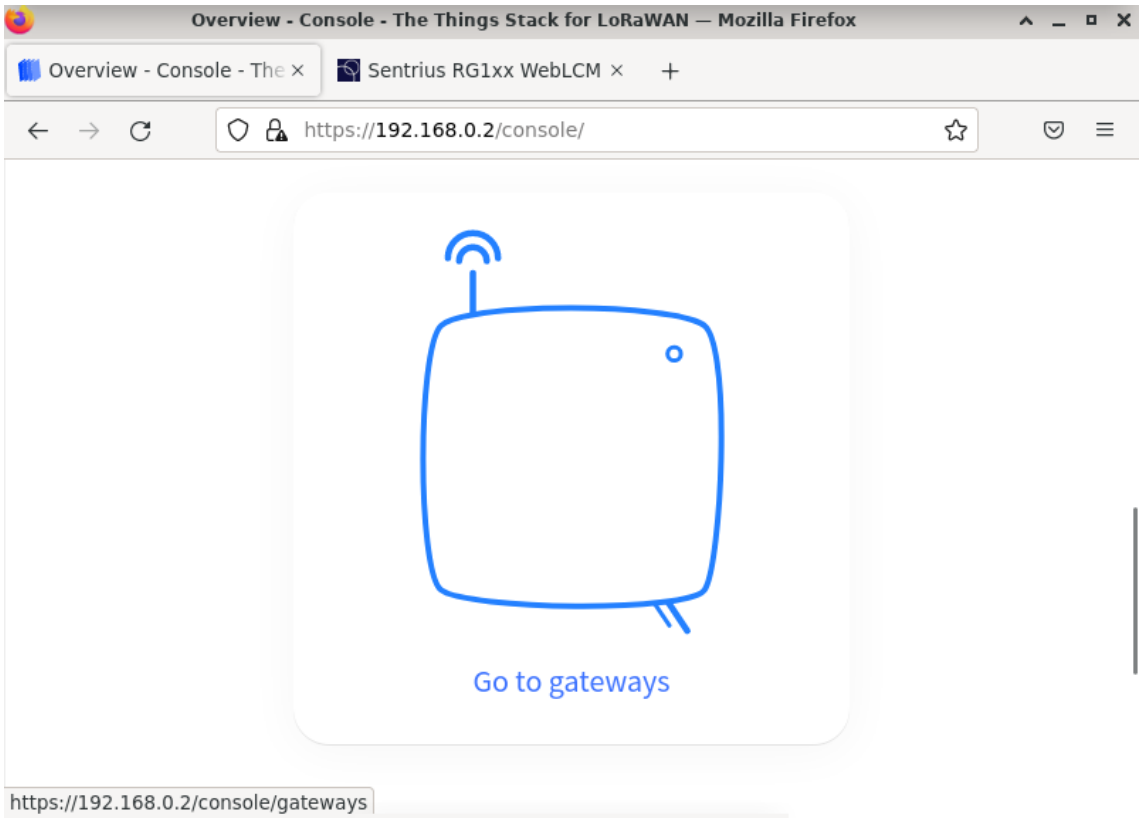


Figura 3.8: Icono de la sección *gateways* en el panel web

Ahora haga clic en registrar una puerta de enlace (Figura 3.9).

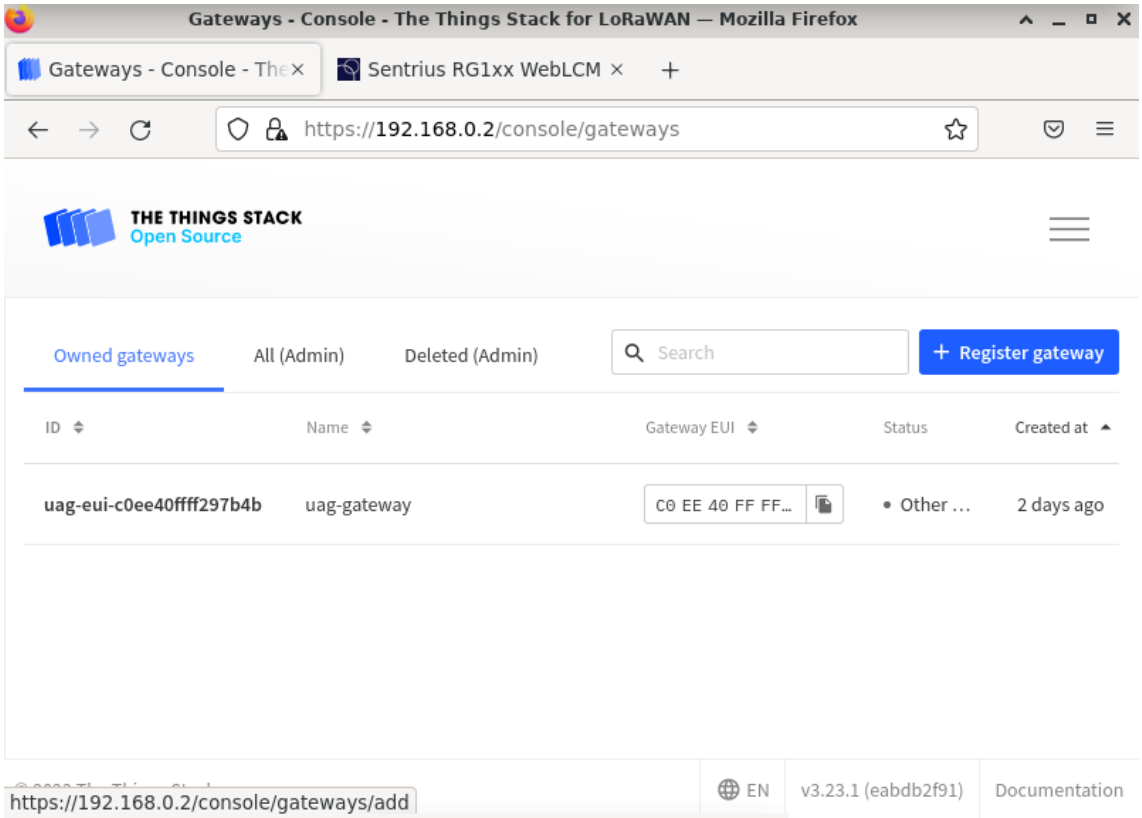


Figura 3.9: Sección con el listado de todos los *gateways* en el panel web

Complete los datos con su EUI correspondiente, ID de puerta de enlace y nombre de puerta de enlace, así como el plan de frecuencia (Figuras 3.10 y 3.11).

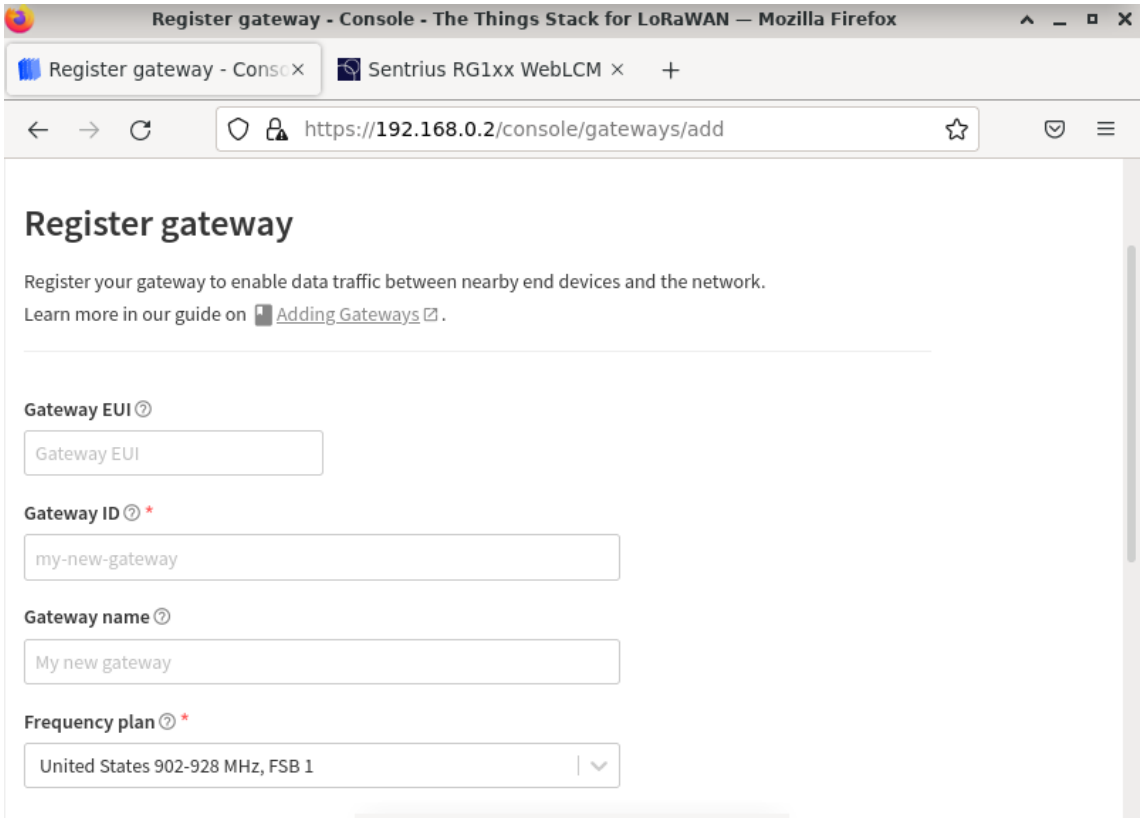


Figura 3.10: Sección superior de configuración para registro de un *gateway*

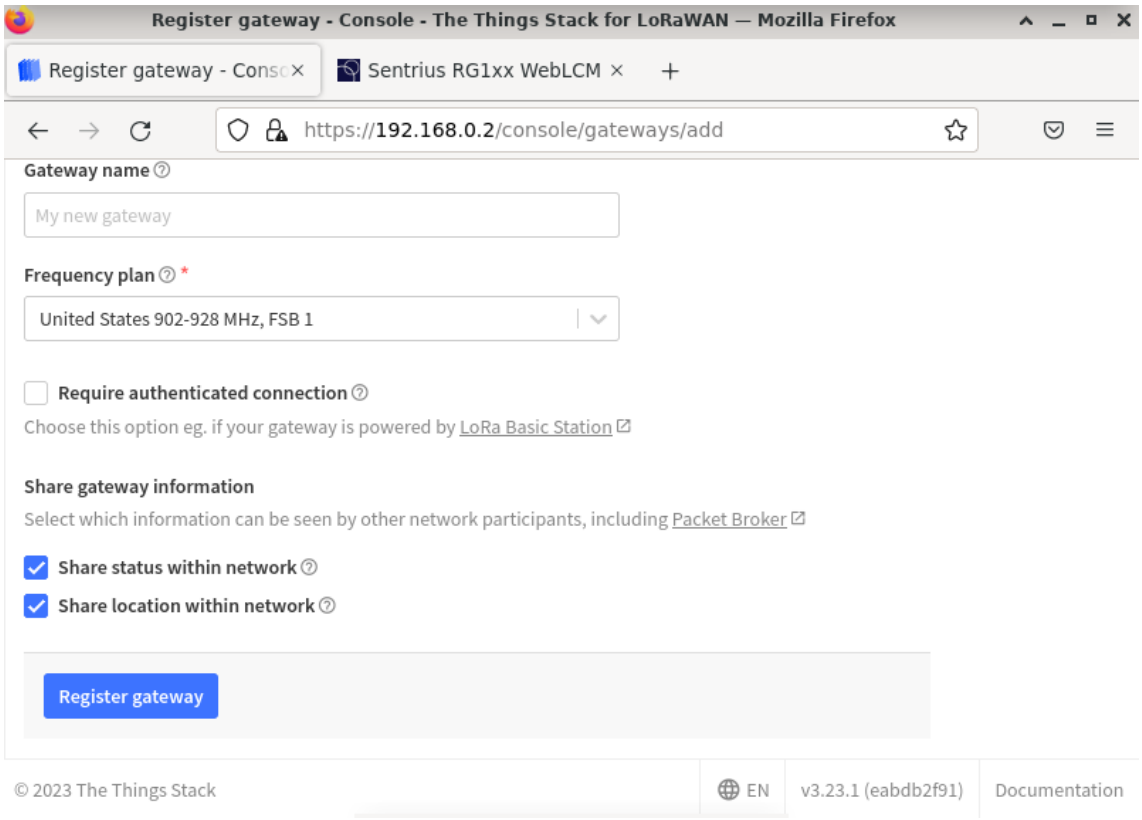


Figura 3.11: Sección inferior de configuración para registro de un *gateway*

Haga clic en registrar puerta de enlace y ahora debería ver una puerta de enlace como esta (Figura 3.12).

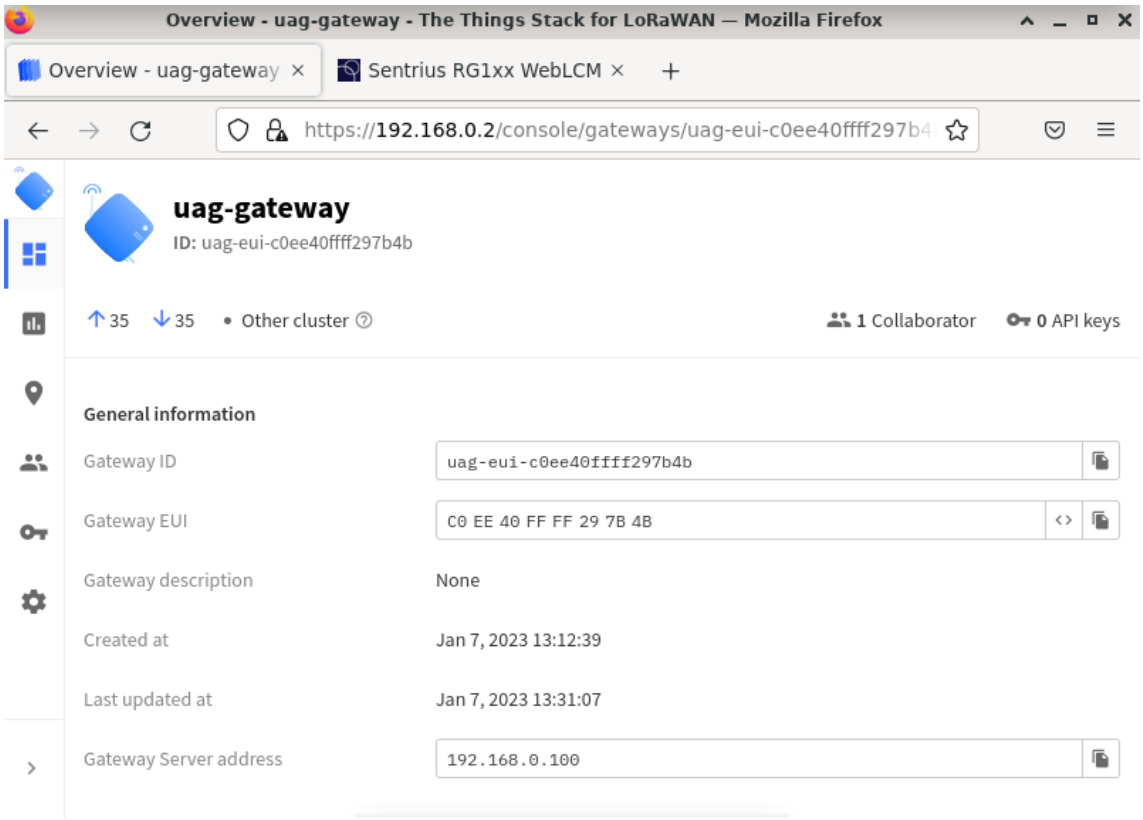


Figura 3.12: Vista de un *gateway* configurado

También recuerde deshabilitar la conexión autenticada en caso de que quiera probar nuevos dispositivos LoRa sin credenciales (Figura 3.13).

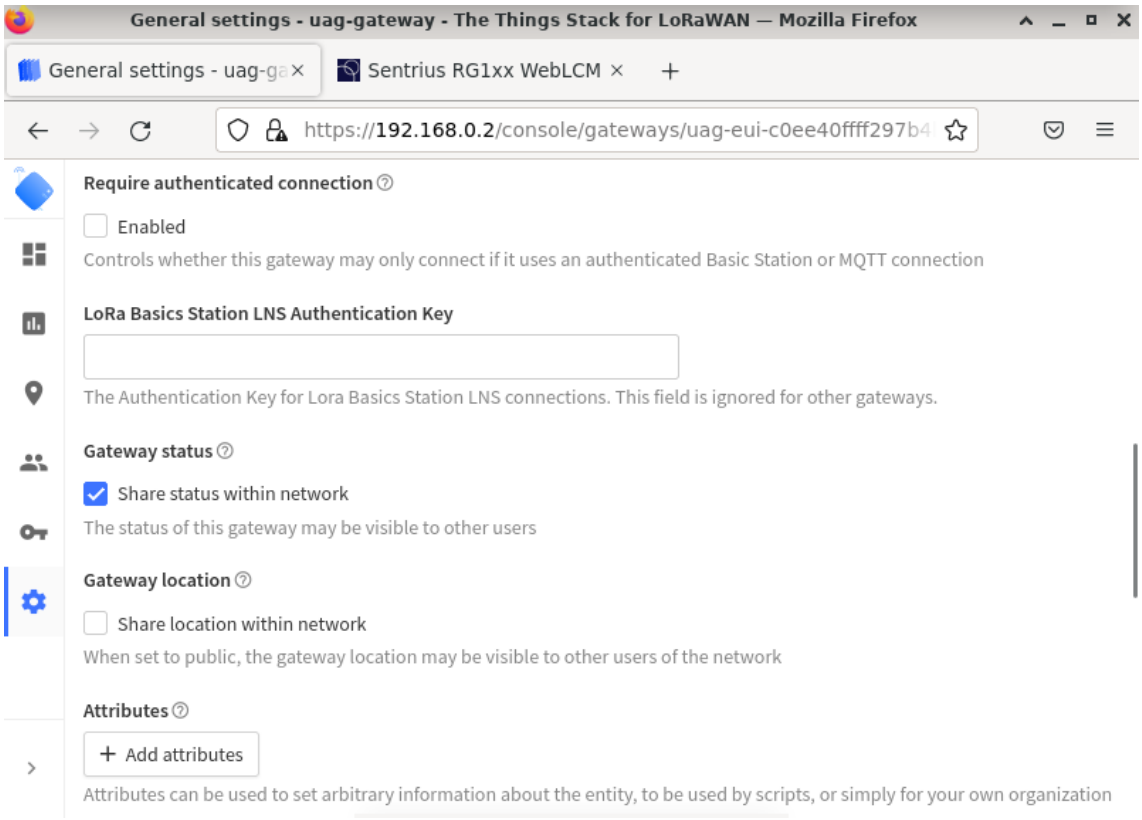


Figura 3.13: Panel para desactivación de autenticación

Ahora puede ver que todo fue exitoso una vez que comenzó a recibir datos en vivo (Figura 3.14) de su puerta de enlace (esto puede demorar hasta 1 o 2 minutos dependiendo de la latencia).

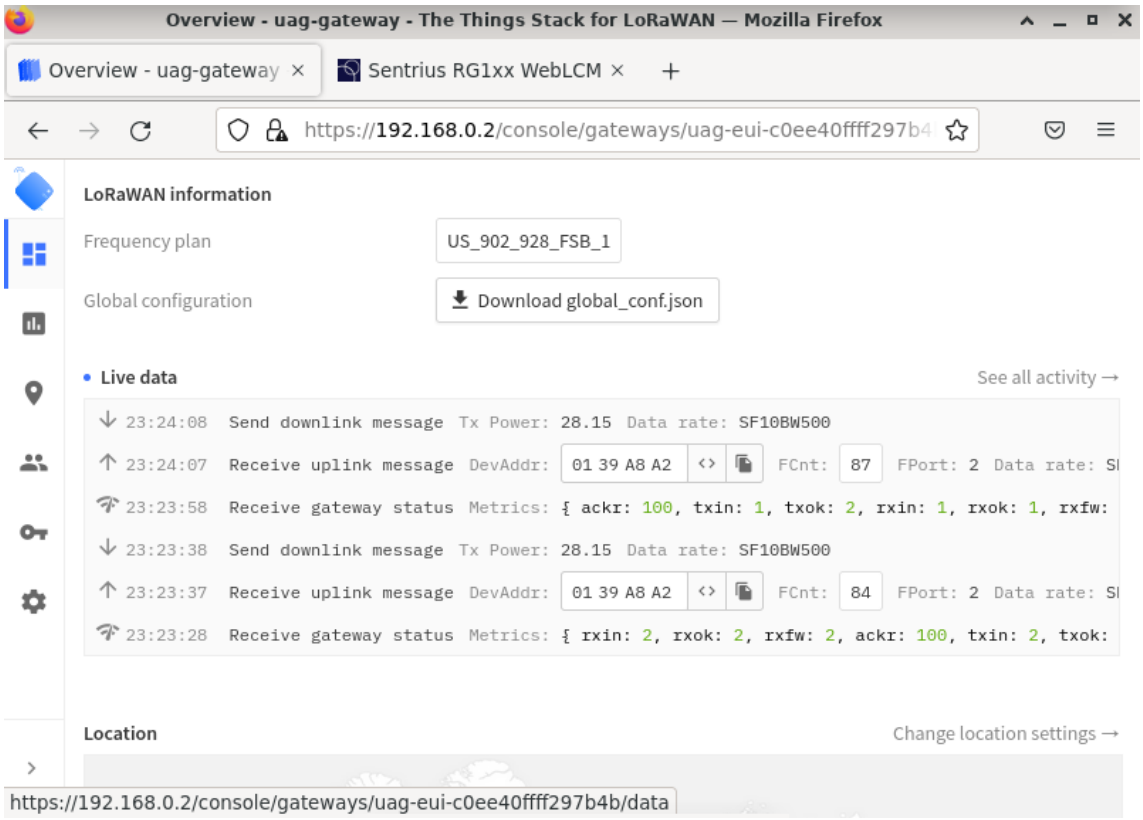


Figura 3.14: Trafico de la red interceptado en el *gateway*

Recuerda que si aún no tienes dispositivos solo verás mensajes administrativos

3.4.2. Crear una aplicación

Ahora estamos listos para configurar una aplicación, esto se usa para encapsular todos los dispositivos en un solo punto y tomar información. Para hacerlo, regrese a la página principal de la consola y haga clic en ir a aplicaciones (Figura 3.15).

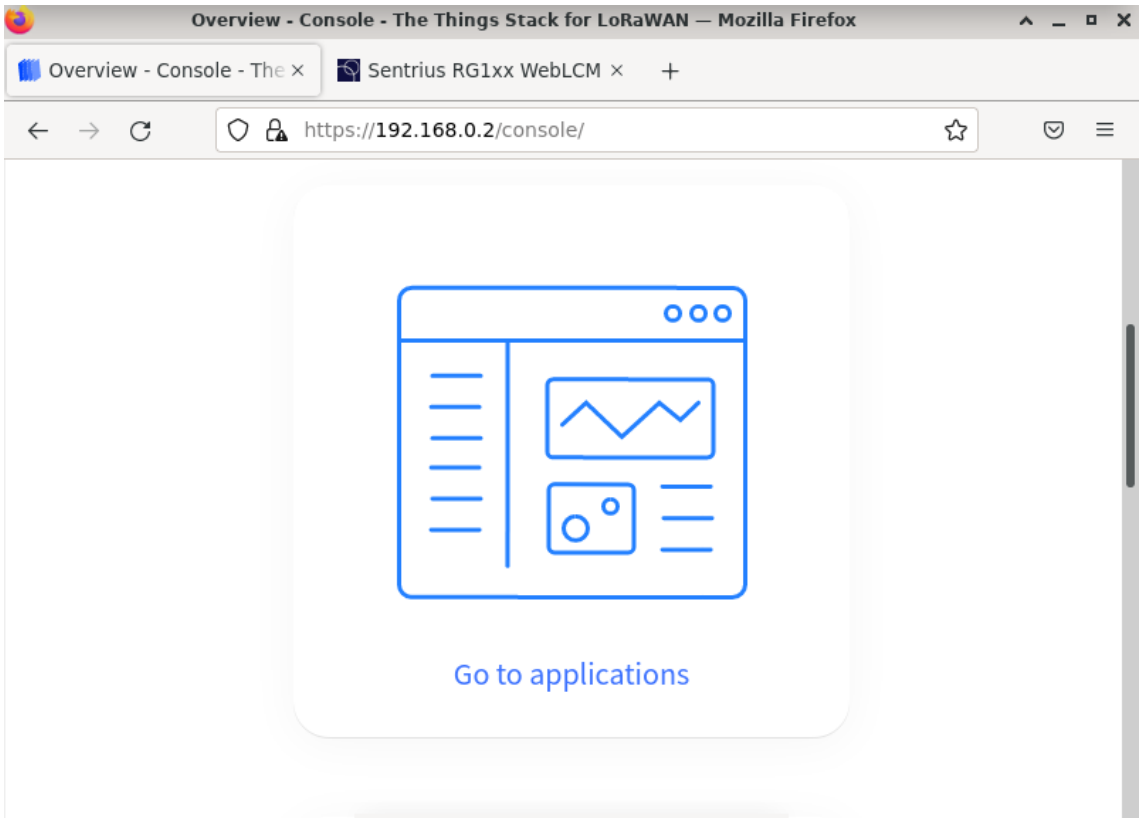


Figura 3.15: Icono de la sección *applications* en el panel web

Haga clic en crear una aplicación (Figura 3.16).

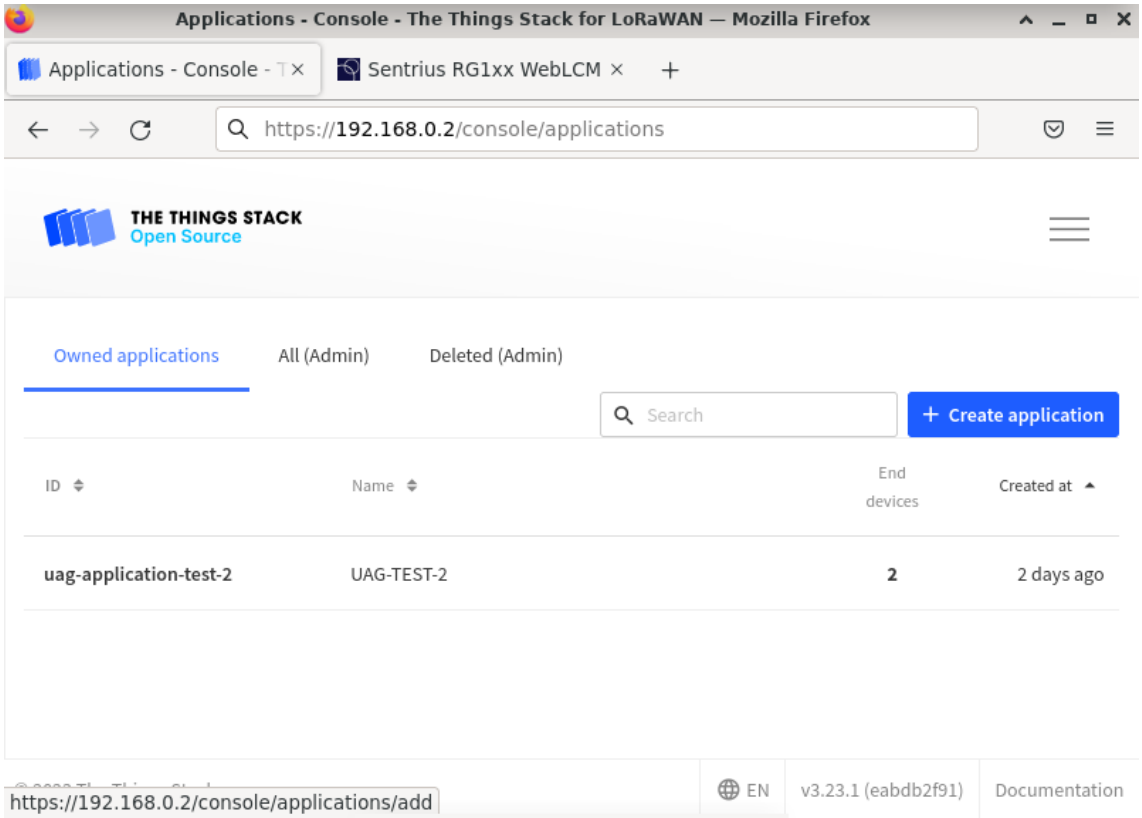


Figura 3.16: Sección con el listado de todos los *applications* en el panel web

Rellénelo con información básica sobre su aplicación y asígnele una identificación única (Figura 3.17).

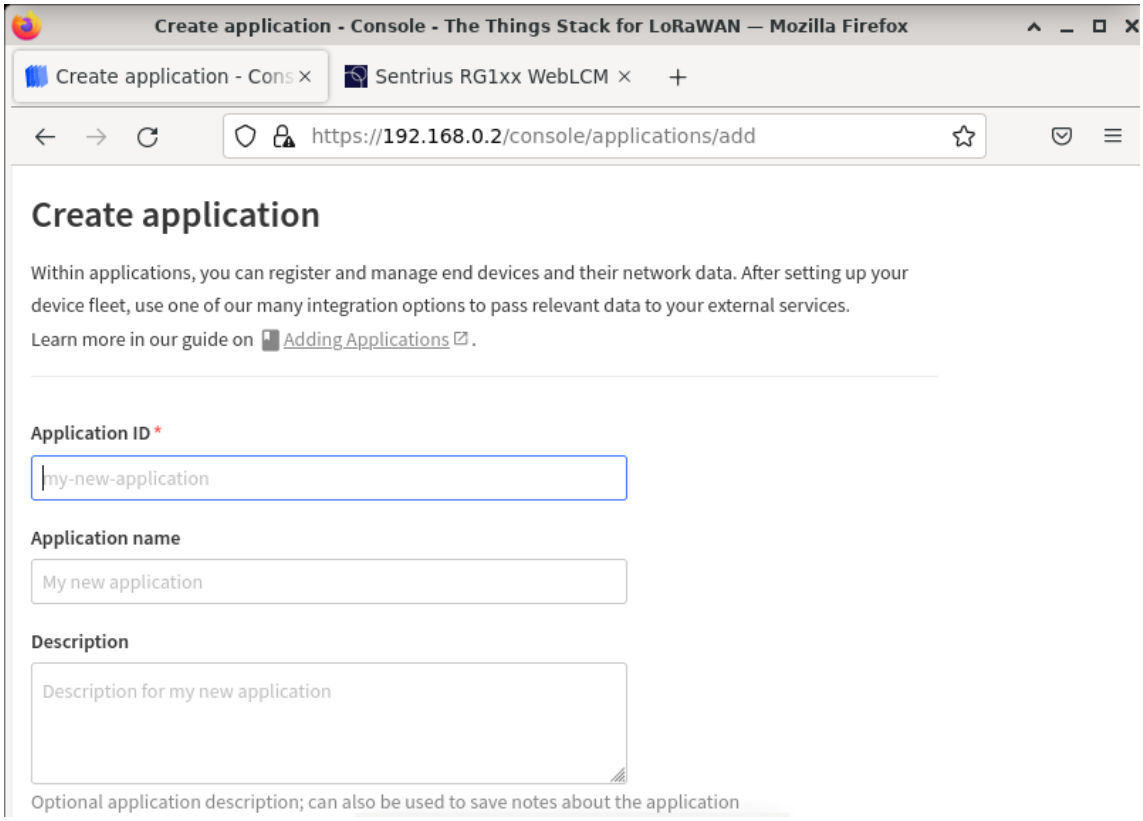


Figura 3.17: Sección de configuración para registro de un *application*

Una vez hecho esto, será redirigido a la página de su aplicación (Figura 3.18).

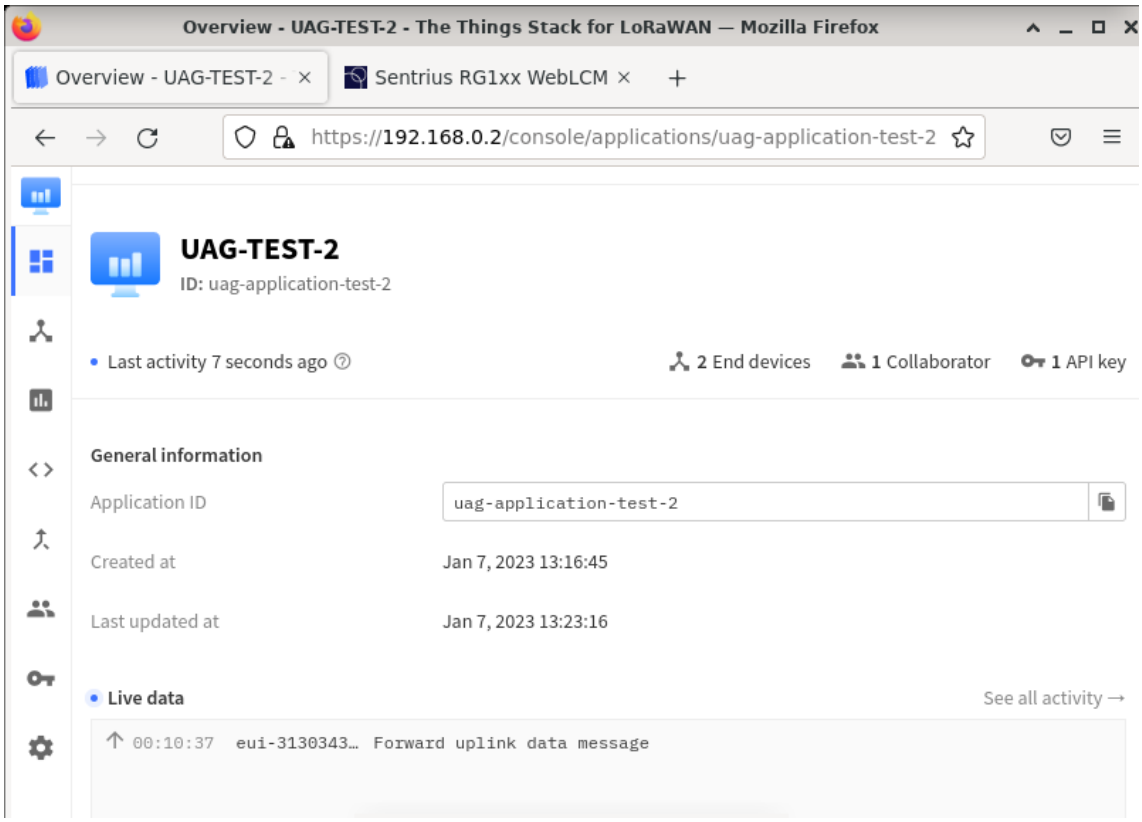


Figura 3.18: Vista de una *application* configurada

3.4.3. Registrar un nuevo dispositivo

Ahora necesitamos agregar algunos nodos o dispositivos a nuestra aplicación para comenzar a capturar datos. Haga clic en Dispositivos finales en la columna izquierda de su aplicación y será redirigido a algo como esto (Figura 3.19).

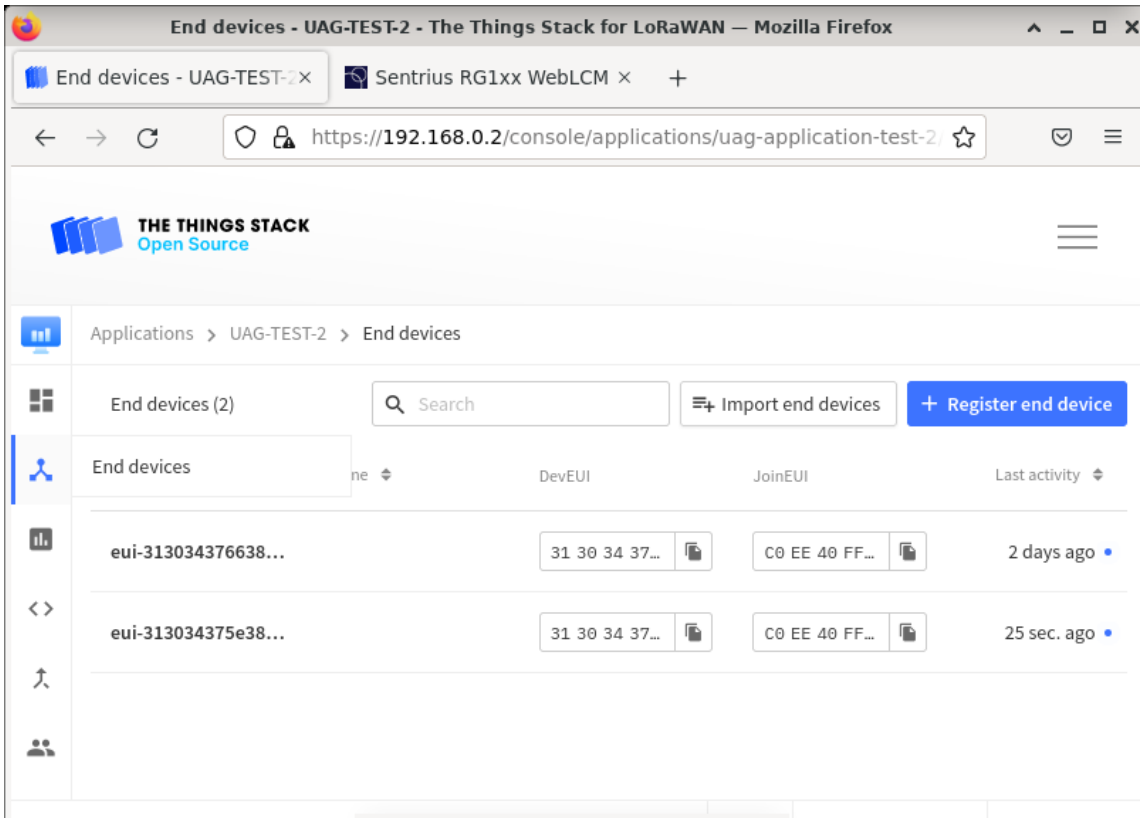


Figura 3.19: Sección superior de configuración para registro de un *device*

Haga clic en Registrar dispositivo final y será redirigido nuevamente al formulario de registro de dispositivo final, haga clic en ingresar manualmente los detalles del dispositivo final (Figura 3.20).

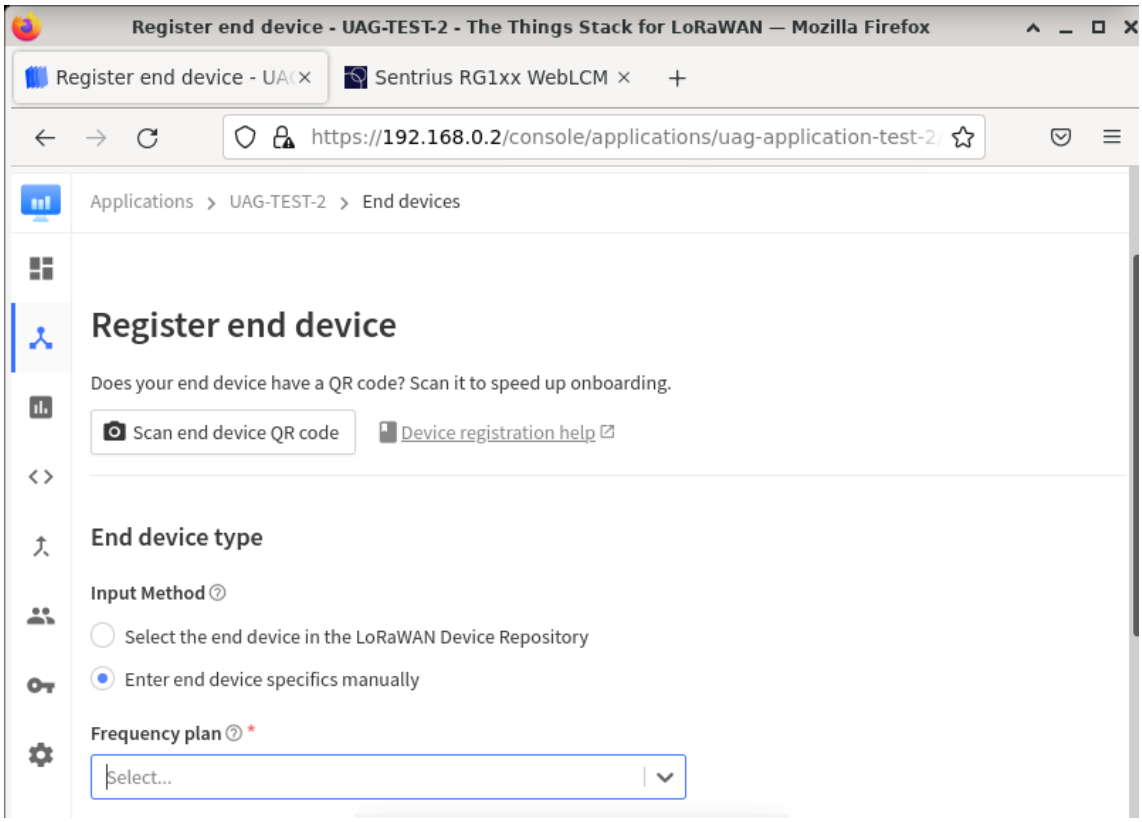


Figura 3.20: Sección superior de configuración para registro de un *device*

Seleccione el plan de frecuencia adecuado y la especificación LoRaWAN (Figura 3.21).

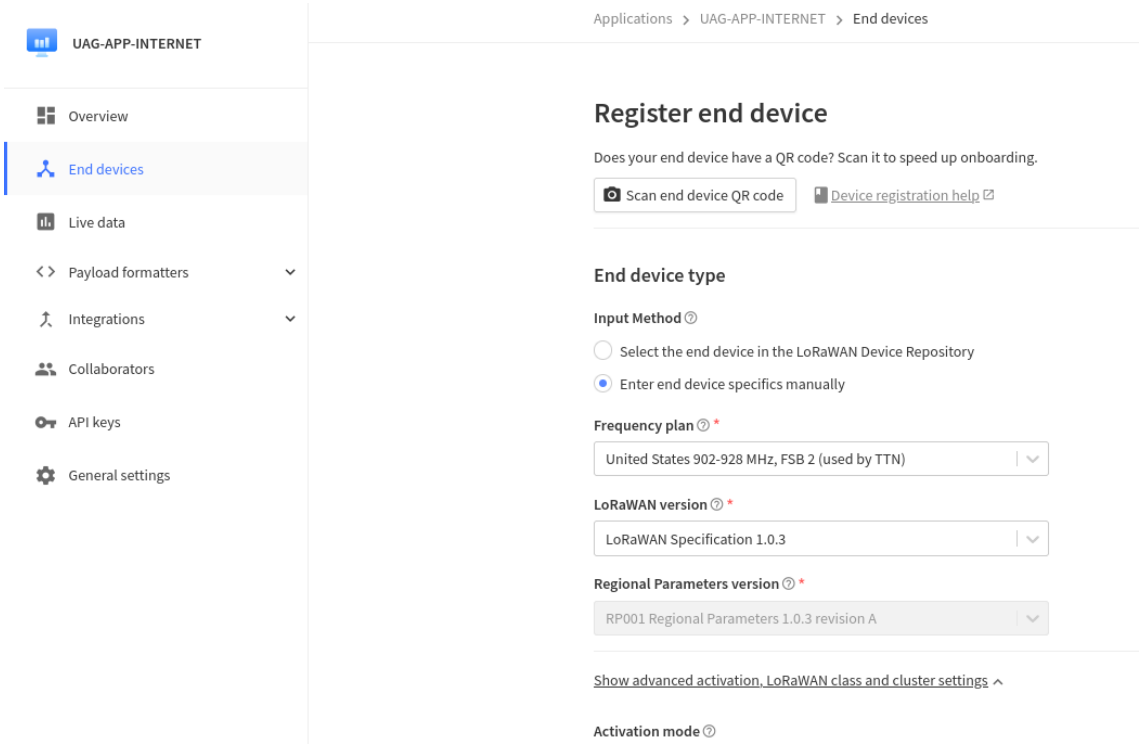


Figura 3.21: Sección de frecuencia de configuración para registro de un *device*

Debido a errores en la implementación, recomendamos el uso de 1.0.3.

Use activación de aire OTAA y clase A únicamente, también mantenga la configuración predeterminada de MAC (Figura 3.22).

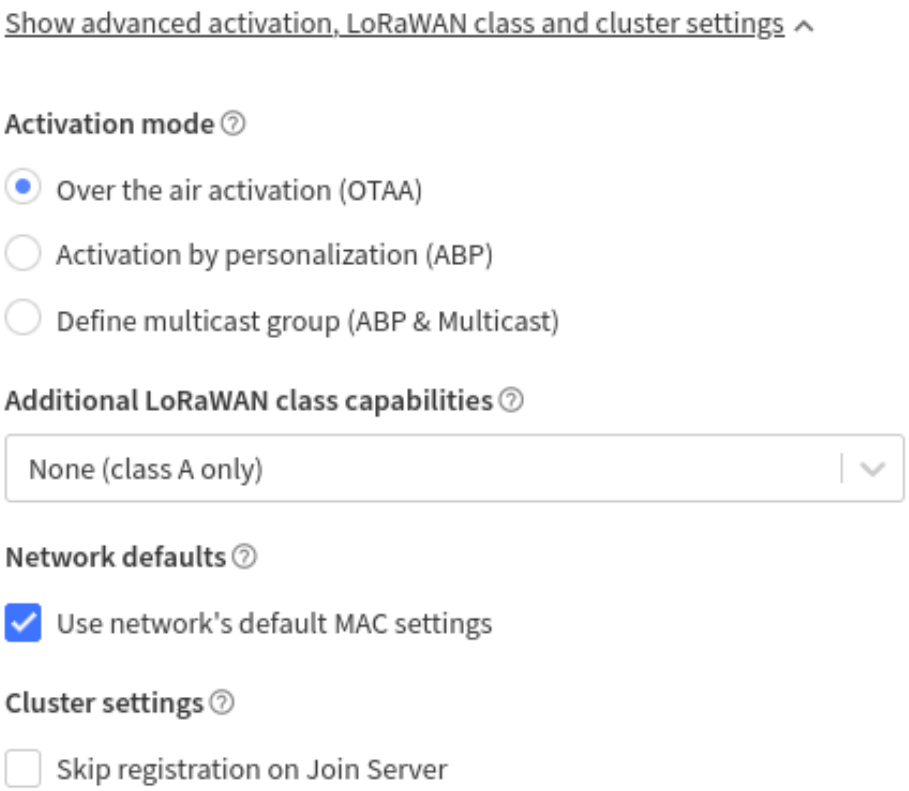


Figura 3.22: Sección media de configuración para registro de un *device*

Agregue JoinEUI, este es básicamente el mismo EUI que viene con su puerta de enlace que se usa para comunicarse en ambos sentidos (Figura 3.23).

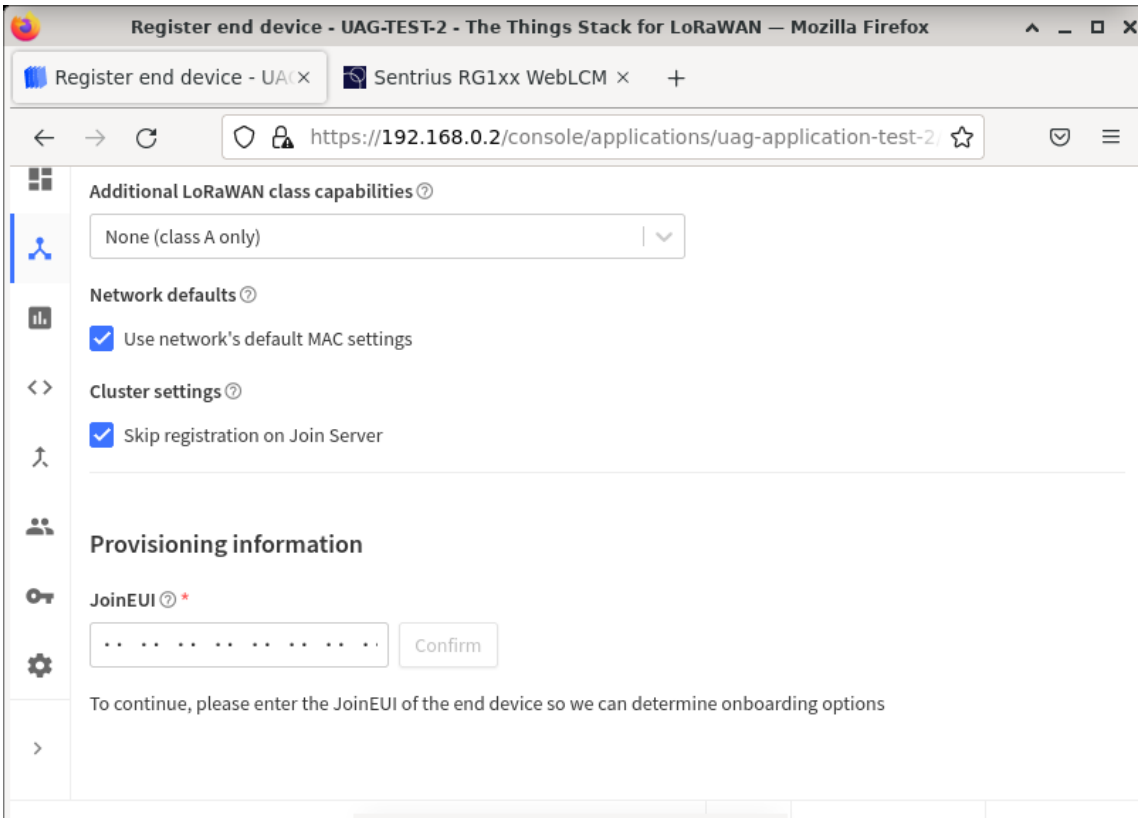


Figura 3.23: Sección inferior de configuración para registro de un *device*

Haga clic en confirmar y ahora podrá ver el dispositivo en su aplicación, así como acceder a él (Figura 3.24).

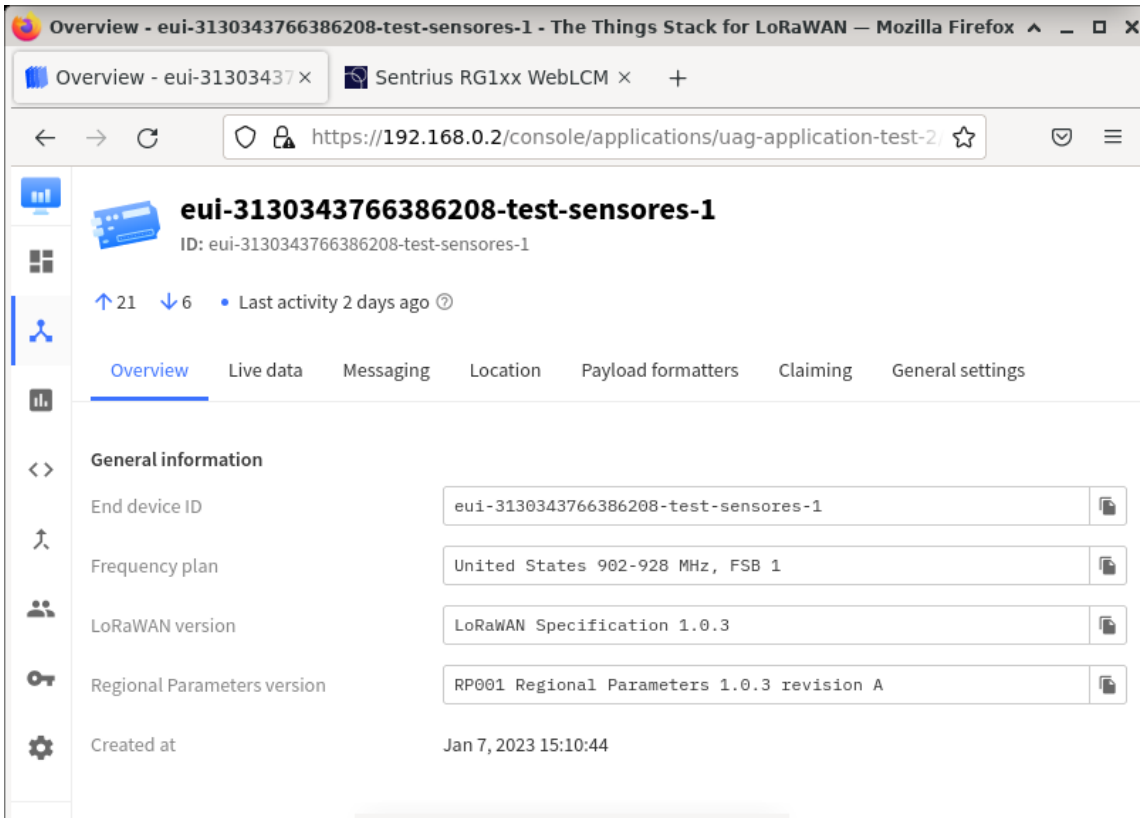


Figura 3.24: Vista de un *device* configurado

Ahora todo está configurado, Conecte su dispositivo a la corriente y comenzará a recibir datos en poco tiempo justo como en la siguiente imagen (Figura 3.25).

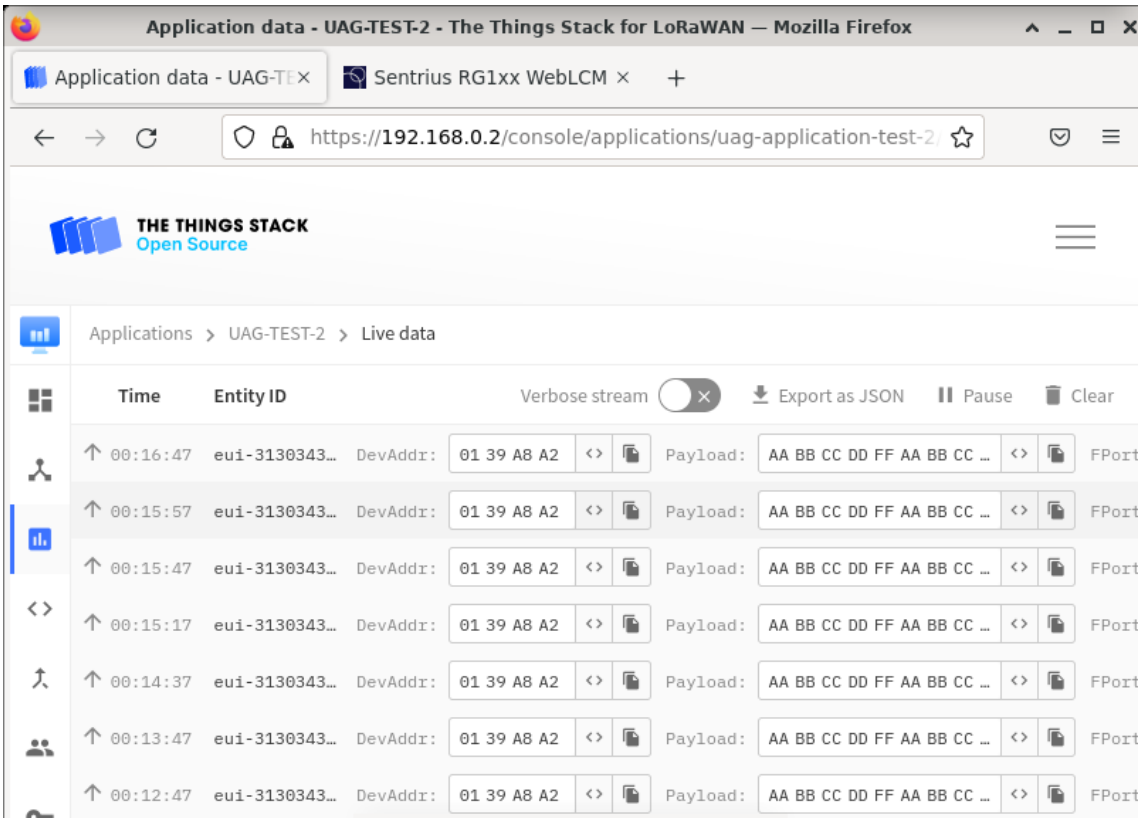


Figura 3.25: Vista de un *device* configurado

3.4.4. Leer información usando Linux

Una vez lanzadas todas las herramientas explicadas en los capítulos anteriores podemos notar que empezamos a recibir información a través del panel web (Figura 3.25) sin embargo podemos extender su funcionalidad leyendo la información utilizando la terminal de comandos. Gracias a esto, podemos tomar generar capas extra de funcionalidad.

La lectura de información del panel web se hace a través de su interfaz de programación de aplicaciones, utilizando el comando curl:

```
curl --cacert cert.pem \
"https://192.168.0.2/api/v3/events" \
-X POST -H "Authorization: Bearer $API_KEY" \
-H "Accept: text/event-stream" \
-H 'Connection: keep-alive' \
--data-raw $'{"identifiers":[{"application_ids":
{"application_id":"uag-application-test-2"}}]}\n'
```

El comando se compone de múltiples opciones.

La primer linea llama el comando curl asi como su certificado *cert.pem* (Generado previamente para el lanzamiento del panel web)

La segunda linea utiliza la dirección IP del panel web con la ruta */api/v3/events*

La tercera linea habla del método de llamada *POST* asi como del uso de la variable *API_KEY* la cual es generada a través del panel web.

La cuarta linea describe el método de comunicación el cual usara un flujo de datos por eventos basados en texto

La quinta linea especifica el mantener la conexión viva y funcionando mientras sigamos necesitando datos.

Por ultimo las dos últimas líneas describen los datos que obtendremos de la interfaz, los cuales son datos de la aplicación *uag-application-test-2*, estos pueden ser modificados consultando la documentación oficial de la interfaz.

Una vez obteniendo los datos obtendremos una cadena de texto tipo JSON, en la cual podremos encontrar múltiples datos relacionados a la llamada entre servidor y nodo, para nuestros propósitos el mas importante es el llamado payload. Este son los datos encriptados del nodo, los cuales están codificados en base64 para su mayor seguridad. Podemos desencriptarlos con alguna herramienta web o utilizando el programa en Python que se vera en la siguiente sección.

3.4.5. Leer información usando código en Python

Una capa extra añadida en nuestra investigación es el uso de un programa automatizado para la recopilación y administración de datos, como prerrequisito necesitamos apoyarnos del comando de Linux que vimos en la sección anterior y enviar los datos a un archivo con el uso del redireccionamiento de datos o con el operador doble mayor que.

$$\gg$$

Una vez que dejemos corriendo este comando empezaremos a recabar datos en un archivo de manera continua y estaremos listos para correr nuestro programa en Python. El cual nos ayuda a separar la información y agregar personalización dependiendo de los requisitos de los usuarios.

El código es el siguiente:

```
#!/usr/bin/env python

"""
This script is used to retrieve last data from server API and send i
to a desired dashboard.

Usage: python middleware.py
"""
import requests
import json
import base64

#INDEXES MEANING
DEVICE_ID   = 0
PAYLOAD     = 1
# URL Where are you storing logs of the server
# Feel free to change to suit your needs
SERVER_LOGS_PATH='/tmp/logs_pako.txt'

# External dashboard URL and headers
# Feel free to modify it to suit your needs
dashboard_url = "https://sistema-incendios-uag.up.railway.app\
/api/nodes-data"
dashboard_headers = {
    'Accept'        : 'application/json',
    #API_KEY
    'Authorization' : 'Bearer 7|yvXruJRQtoWNqsNZYQQdrmLqNj69mIg4ej06F
    'Content-Type'  : 'application/json'
}

def decrypt_data(string):
```

```
    """
    Function to decrypt any data in base64 to binary
    """
    decode_payload = base64.b64decode(string.encode('ascii'))
    return decode_payload.hex()

def call_node_data():
    """
    Read the logs from the watcher
    """
    with open(SERVER_LOGS_PATH, 'r') as f:
        last_line = f.readlines()[-3]
    response = json.loads(last_line)
    #Get the payload
    return [response['result']['identifiers'][0]['device_ids']['device

def format_payload(payload, device_id):
    """
    Here we can format the payload to our needs
    """
    #IMPORTANT for this example we are sending all the PAYLOAD as
    #TEMPERATURE it dependes on programmer to split data as needed
    dashboard_payload={
        "node_id": device_id,
        "temperature" : payload,
        # "gas": 69,
        # "humidity": 666,
        # "fire": 420
    }
    return dashboard_payload

def submit_node_data(data):
```

```
    """
    Submit node data to dashboard
    """
    response = requests.post(dashboard_url,
    json=data, headers=dashboard_headers)
    if response.status_code != 200 | response.status_code != 201:
        print('ERROR: {}. Status code: {}'.format(
        response.reason, response.status_code))
        print('ERROR: {}'.format(response.text))
    return response

if __name__ == '__main__':
    #TODO: add persistant running with while and keep
    #last line if changed run cycle again, if not skip until
    #next minute or whatever time we want

    # Get data from nodes
    response = call_node_data()
    # Decrypt payload
    raw_payload = decrypt_data(response[PAYLOAD])
    # Format payload
    dashboard_payload = format_payload(raw_payload,
    response[DEVICE_ID])
    # Print message to be sent
    print(dashboard_payload)
    # Send the message to dashboard
    submit_node_data(dashboard_payload)
```

El programa se compone de cuatro funciones principales.

La primera función *call_ node_ data* hace el llamado del archivo donde guardamos los datos, leyendo las ultimas líneas y transformándolas en un objeto JSON para extraer los datos del nodo.

La segunda función *decrypt_ data* nos ayuda a desencriptar los datos en base64

de forma automática, obteniendo una trama legible en texto plano.

Mas adelante la tercera función *dashboard_ payload* nos ayuda a formar una nueva trama la cual será enviada por la función *submit_ node_ data*

3.4.6. Problemas comunes encontrados y sus soluciones

P: ¿Qué debo hacer para instalar Debian sin internet en el servidor?

R: Como el servidor está aislado sin una interfaz Wi-Fi, recomendamos descargar una imagen de CD/DVD, que tiene más memoria, pero la ventaja es que viene con muchas herramientas preconstruidas instaladas como un entorno de escritorio.

P: No puedo conectarme a la URL de API, ¿qué puedo hacer?

R: A partir del 11 de enero de 2023, existe un problema conocido con respecto al archivo ttn, agregue este código a continuación en console.ui Verifique el archivo del repositorio como referencia

```
dcs:
  base-url: "https://192.168.0.2/api/v3"
```

P: Al instalar Go hay un error con una variable llamada GOPATH que no me deja continuar, ¿qué puedo hacer?

R: Si se configura GOPATH, se producirá un error al eliminar esta variable.

```
unset GOPATH
```

P: Al instalar Go, hay un error con respecto al comando make.

R: En caso de que no tenga el comando make, instale las herramientas esenciales en Debian, el comando es

```
sudo apt-get install build-essential
```

P: No puedo iniciar el tablero usando Redactar debido a un puerto de error

R: A veces, otros servicios toman algunos puertos necesarios para que el tablero funcione para verificar qué servicios están actualmente tomando puertos y ejecutándose, use el siguiente comando

```
sudo ss -nlpt
```

En nuestro caso, apache2 estaba tomando el puerto 80, por lo que necesitamos detenerlo.

```
sudo service apache2 stop
```

P: Completé la información de la puerta de enlace pero no se registró

R: A veces, la información de la puerta de enlace no se registra como se esperaba y tenemos que modificarla manualmente. Para hacerlo, vaya a la configuración y modifique el campo de dirección del servidor de la puerta de enlace (Figura 3.26).

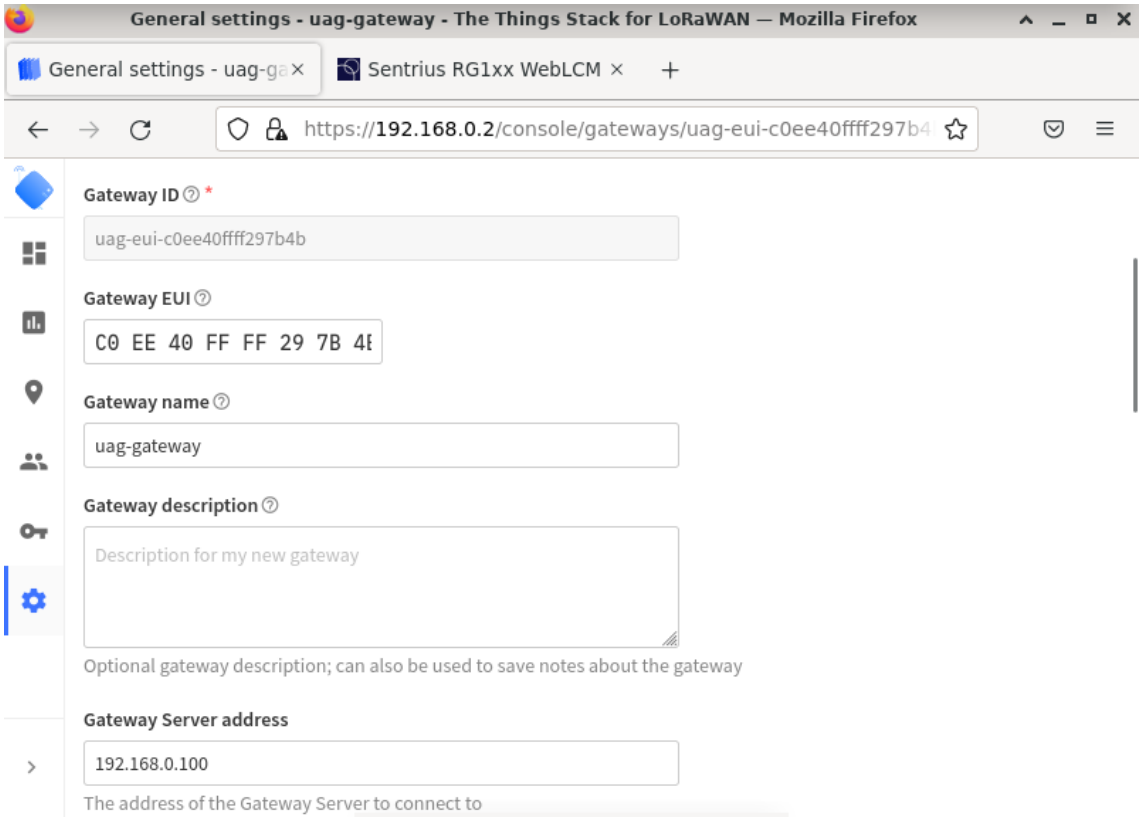


Figura 3.26: Vista de configuración para la IP de un *gateway*

P: No puedo entender que la carga útil parece estar encriptada

R: Puede deshabilitar los formateadores de la carga útil para ver los datos sin formato en caso de que quiera probar. Vaya a formateadores de carga útil en su dispositivo y seleccione Ninguno (Figura 3.27).

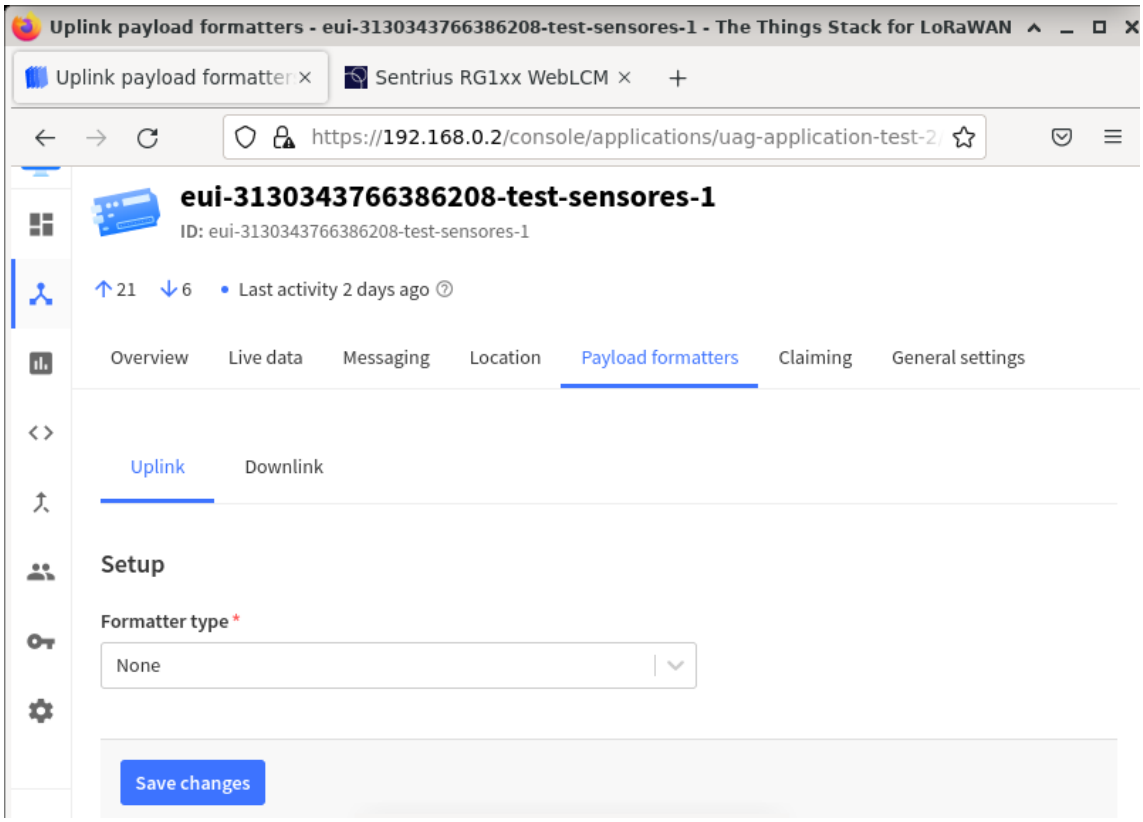


Figura 3.27: Vista para el formato de los paquetes

Guardar los cambios

P: La aplicación no me permite registrar un dispositivo porque ya está en uso

R: Cambie el EUI de su dispositivo y cambie la ID del dispositivo una vez que se registre en la aplicación

P: No puedo revisar los mensajes de datos en vivo en el tablero

R: A veces, este problema se debe a que la puerta de enlace no puede comunicarse con los nodos, verifique dos veces los EUI de la puerta de enlace y los dispositivos e intente volver a conectarse con la puerta de enlace usando el comando ping a la IP.

Capítulo 4

Conclusiones

4.1. Conclusiones y las recomendaciones

En este ultimo capitulo se habla sobre las conclusiones obtenidas durante la investigación, su desarrollo, logros conseguidos y proyectos futuros que puedan surgir de este.

Descripción

En este proyecto se ha diseñado todo un flujo de trabajo que hace uso de la tecnología LoRaWAN, se ha buscado la generalización par que distintos usuarios, investigadores o empresas puedan usarlo en sus monitoreos y obtener distintas métricas asi como tomar acciones en base a estos datos. Además de ello, los programas desarrollados propuestos pueden ser escalables a distintos tipos de funcionalidades e inclusive tecnologías.

Como primer punto, se hizo una investigación de la tecnología LoRa y LoRaWAN y se resalto el como esta puede funcionar en equipos mínimos a grandes distancias utilizando pocos recursos. Se procedió al desarrollo conjunto de una solución embebida al mismo tiempo que se montaba y configuraba un servidor maestro que puede ser escalable a múltiples servidores con múltiples nodos. Una vez teniendo todo configurado todo el sistema se pasó a la programación de una capa superior para la especialización de estos nodos por dos equipos de investigadores distintos demostrando asi su capacidad de adaptación para futuros proyectos con el minimo esfuerzo.

Teniendo todo el trabajo se pasó al paso de pruebas de todo el flujo de trabajo

obteniendo datos desde múltiples nodos siendo procesados y enviados a los paneles web para finalmente ser desplegados apropiadamente, demostrando de esta forma la robustez del sistema. Además de la documentación de todos estos procedimientos no solo en este trabajo si no en un repositorio privado para los investigadores involucrados en el proyecto.

Trabajos futuros

A pesar de la robustes del flujo de trabajo, hay cabida para múltiples puntos de mejora o distintas soluciones extras y capas de funcionalidad que pueden ser implementadas en base a distintos requisitos.

- Montaje de un servidor de servidores en la nube.
- Orquestación de contenedores asi como balanceo de cargas entre ellos.
- Análisis del rendimiento y coste energético del sistema para la búsqueda de optimización en estas áreas.
- Implementar una solución más compacta para continuar simplificando su funcionalidad en usuarios finales.

Bibliografía

[1] M. Dong, H. Yu, Z. Sun, M. Wu, L. Zhang, Y. Sui, G. Yu, T. Han, and R. Zhao, "Design of iot gateway for crop growth environmental monitoring based on edge-computing technology autor," Master's thesis, Mudanjian Medical University, Heilongjiang China, jul 2022.

[2] E. Karantoumanis, X. Krallis, and N. Ploskas, "An experimental evaluation of sensors on a lorawan-based gps tracking system," Master's thesis, University of Western Macedonia, Greece, feb 2022.

[3] Z. Gere, "Implementation of lorawan from end-device to application," Master's thesis, Metropolia University of Applied Sciences, nov 2021.

[4] M. Cimdins, F. John, and H. Hellbruck, "Flexible data acquisition with lora-wan and mqtt for small and medium-sized enterprises," Master's thesis, Mobile Communication - Technologies and Applications; 25th ITG-Symposium, dec 2021.

[5] P. A. Barro, M. Zennaro, and E. Pietrosemoli, "The local things network: on the design of a lorawan gateway with autonomous servers for disconnected communities," Master's thesis, 2019 Wireless Days, apr 2019.

[6] M. T. Antúnez, "Particionado de red de acceso radio en redes lorawan," Master's thesis, Escuela Técnica Superior de Ingenierías de Informática y Telecomunicación, jun 2022.

[7] J. A. R. Sánchez and J. S. R. González, "Diseño de un nodo lora-gps para localización de bicicletas implementado en una red lorawan," Master's thesis, UNIVERSIDAD SANTO TOMÁS, Bogota, 2021.

[8] M. S. Mahmoud and A. A. H. Mohamad, "A study of efficient power consumption wireless communication techniques / modules for internet of things (iot) applications," Master's thesis, Advances in Internet of Things, Vol. 6, No. 2, apr 2016.

[9] E. Murdyantoro, A. W. W. Nugraha, A. W. Wardhana, A. Fadli, and M. I. Zulfa, "A review of lora technology and its potential use for rural development in indonesia," in *AIP Conference Proceedings*, vol. 2094, p. 020011, AIP Publishing LLC, 2019.

[10] B. B. Rad, H. J. Bhatti, and M. Ahmadi, "An introduction to docker and analysis of its performance," *International Journal of Computer Science and Network Security (IJCSNS)*, vol. 17, no. 3, p. 228, 2017.

[11] M. Torres Padín, "A lora-based framework with applications to smart buildings," Master's thesis, Universitat Politècnica de Catalunya, 2020.